

untitled23-2

November 4, 2025

1 Import necessary libraries

```
[ ]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')
```

2 Import all the necessary dataset

```
[ ]: zomato=pd.read_csv("Zomato Dataset.csv")
d=zomato
```

3 Visualizing the dataset

```
[ ]: d.head()
```

```
[ ]: Restaurant_Name Dining_Rating Delivery_Rating Dining_Votes \
0 Doner King 3.9 4.2 39
1 Doner King 3.9 4.2 39
2 Doner King 3.9 4.2 39
3 Doner King 3.9 4.2 39
4 Doner King 3.9 4.2 39

Delivery_Votes Cuisine Place_Name City Item_Name \
0 0 Fast Food Malakpet Hyderabad Platter Kebab Combo
1 0 Fast Food Malakpet Hyderabad Chicken Rumali Shawarma
2 0 Fast Food Malakpet Hyderabad Chicken Tandoori Salad
3 0 Fast Food Malakpet Hyderabad Chicken BBQ Salad
4 0 Fast Food Malakpet Hyderabad Special Doner Wrap Combo

Best_Seller ... Is_Bestseller Restaurant_Popularity \
0 BESTSELLER ... 1 46
1 BESTSELLER ... 1 46
```

2	NONE	...	1	46
3	BESTSELLER	...	1	46
4	MUST TRY	...	1	46

	Avg_Rating_Restaurant	Avg_Price_Restaurant	Avg_Rating_Cuisine	\
0	4.05	171.347826	3.81868	
1	4.05	171.347826	3.81868	
2	4.05	171.347826	3.81868	
3	4.05	171.347826	3.81868	
4	4.05	171.347826	3.81868	

	Avg_Price_Cuisine	Avg_Rating_City	Avg_Price_City	Is_Highly_Rated	\
0	255.954894	3.939031	245.130482	1.0	
1	255.954894	3.939031	245.130482	1.0	
2	255.954894	3.939031	245.130482	1.0	
3	255.954894	3.939031	245.130482	1.0	
4	255.954894	3.939031	245.130482	1.0	

	Is_Expensive
0	0.0
1	0.0
2	0.0
3	0.0
4	0.0

[5 rows x 26 columns]

```
[ ]: d.tail()
```

```
[ ]:
      Restaurant_Name  Dining_Rating  Delivery_Rating  Dining_Votes  \
57416  Arabian Dreams           2.9           3.6           11
57417  Arabian Dreams           2.9           3.6           11
57418  Arabian Dreams           2.9           3.6           11
57419  Arabian Dreams           2.9           3.6           11
57420  Arabian Dreams           2.9           3.6           11
```

	Delivery_Votes	Cuisine	Place_Name	City	\
57416	0	Shawarma	Kadavanthra	Kochi	
57417	0	Shawarma	Kadavanthra	Kochi	
57418	0	Shawarma	Kadavanthra	Kochi	
57419	0	Shawarma	Kadavanthra	Kochi	
57420	0	Shawarma	Kadavanthra	Kochi	

	Item_Name	Best_Seller	...	Is_Bestseller	\
57416	Kuzhi Mandi	BESTSELLER	...	1	
57417	Vegetable Noodles	NONE	...	1	
57418	Vegetable Schezwan Noodles	NONE	...	1	

57419	Egg Noodles	NONE	...	1
57420	Egg Schezwan Noodles	NONE	...	1

	Restaurant_Popularity	Avg_Rating_Restaurant	Avg_Price_Restaurant	\
57416	145	3.25	158.813793	
57417	145	3.25	158.813793	
57418	145	3.25	158.813793	
57419	145	3.25	158.813793	
57420	145	3.25	158.813793	

	Avg_Rating_Cuisine	Avg_Price_Cuisine	Avg_Rating_City	Avg_Price_City	\
57416	3.406122	164.52551	3.869203	225.95422	
57417	3.406122	164.52551	3.869203	225.95422	
57418	3.406122	164.52551	3.869203	225.95422	
57419	3.406122	164.52551	3.869203	225.95422	
57420	3.000000	NaN	NaN	NaN	

	Is_Highly_Rated	Is_Expensive
57416	0.0	0.0
57417	0.0	0.0
57418	0.0	0.0
57419	0.0	0.0
57420	NaN	NaN

[5 rows x 26 columns]

```
[ ]: d.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 57421 entries, 0 to 57420
```

```
Data columns (total 26 columns):
```

#	Column	Non-Null Count	Dtype
---	-----	-----	-----
0	Restaurant_Name	57421 non-null	object
1	Dining_Rating	57421 non-null	float64
2	Delivery_Rating	57421 non-null	float64
3	Dining_Votes	57421 non-null	int64
4	Delivery_Votes	57421 non-null	int64
5	Cuisine	57421 non-null	object
6	Place_Name	57421 non-null	object
7	City	57421 non-null	object
8	Item_Name	57421 non-null	object
9	Best_Seller	57421 non-null	object
10	Votes	57421 non-null	int64
11	Prices	57421 non-null	float64
12	Average_Rating	57421 non-null	float64
13	Total_Votes	57421 non-null	int64

```

14 Price_per_Vote      57421 non-null float64
15 Log_Price          57421 non-null float64
16 Is_Bestseller      57421 non-null int64
17 Restaurant_Popularity 57421 non-null int64
18 Avg_Rating_Restaurant 57421 non-null float64
19 Avg_Price_Restaurant 57421 non-null float64
20 Avg_Rating_Cuisine  57421 non-null float64
21 Avg_Price_Cuisine   57420 non-null float64
22 Avg_Rating_City     57420 non-null float64
23 Avg_Price_City      57420 non-null float64
24 Is_Highly_Rated     57420 non-null float64
25 Is_Expensive        57420 non-null float64
dtypes: float64(14), int64(6), object(6)
memory usage: 11.4+ MB

```

```
[ ]: d.dtypes
```

```

[ ]: Restaurant_Name      object
Dining_Rating            float64
Delivery_Rating          float64
Dining_Votes             int64
Delivery_Votes           int64
Cuisine                  object
Place_Name               object
City                     object
Item_Name                object
Best_Seller              object
Votes                    int64
Prices                   float64
Average_Rating           float64
Total_Votes              int64
Price_per_Vote           float64
Log_Price                float64
Is_Bestseller            int64
Restaurant_Popularity    int64
Avg_Rating_Restaurant    float64
Avg_Price_Restaurant     float64
Avg_Rating_Cuisine       float64
Avg_Price_Cuisine        float64
Avg_Rating_City          float64
Avg_Price_City           float64
Is_Highly_Rated          float64
Is_Expensive             float64
dtype: object

```

```
[ ]: d.shape
```

```
[ ]: (57421, 26)
```

```
[ ]: d['City'].nunique()
```

```
[ ]: 6
```

```
[ ]: d['City'].value_counts()
```

```
[ ]: City
      Hyderabad    15613
      Mumbai      11729
      Chennai     11705
      Jaipur       11235
      Pune         6450
      Kochi        689
      Name: count, dtype: int64
```

```
[ ]: d['Avg_Price_City'].nunique()
```

```
[ ]: 6
```

```
[ ]: d['Avg_Price_City'].value_counts()
```

```
[ ]: Avg_Price_City
      245.130482    15613
      304.402778    11729
      261.863993    11705
      217.163255    11235
      242.170568     6450
      225.954220     688
      Name: count, dtype: int64
```

```
[ ]: d['Average_Rating'].nunique()
```

```
[ ]: 41
```

```
[ ]: d['Average_Rating'].value_counts()
```

```
[ ]: Average_Rating
      4.000000    4951
      3.950000    4107
      4.100000    4039
      4.150000    3332
      3.911132    2730
      4.050000    2708
      3.850000    2554
      3.961132    2510
```

3.861132	2345
3.600000	2189
3.800000	1931
4.011132	1926
4.200000	1895
3.900000	1833
3.750000	1751
3.650000	1561
3.711132	1430
3.700000	1162
3.550000	1126
3.811132	1097
3.761132	900
4.350000	863
4.061132	858
4.250000	851
4.300000	842
3.450000	637
3.250000	581
3.611132	520
3.200000	455
4.111132	451
3.300000	428
3.500000	421
3.350000	420
4.400000	420
3.561132	372
3.892724	273
4.450000	258
3.511132	200
3.661132	193
4.161132	162
3.400000	139

Name: count, dtype: int64

```
[ ]: d['Avg_Price_Restaurant'].nunique()
```

```
[ ]: 410
```

```
[ ]: d['Avg_Price_Restaurant'].value_counts()
```

```
[ ]: Avg_Price_Restaurant
```

280.308571	700
346.912605	595
354.201802	555
336.678899	545
355.054724	508

```

...
150.000000    9
93.000000     5
126.000000    5
428.000000    5
352.382500    4
Name: count, Length: 410, dtype: int64

```

```
[ ]: d['Avg_Price_Cuisine'].nunique()
```

```
[ ]: 39
```

```
[ ]: d['Avg_Price_Cuisine'].value_counts()
```

```
[ ]: Avg_Price_Cuisine
254.565364    22738
260.363492     5503
255.954894     5271
254.494549     4174
232.318824     2786
266.881510     2573
212.572315     1466
188.819633     1351
170.564668     1221
273.985376     1070
188.590959       899
208.939511       814
296.597212       812
256.072535       697
252.081731       686
318.322826       516
349.923944       508
223.146488       485
239.902978       478
187.315722       379
213.796713       324
240.013763       298
264.894330       292
187.788874       251
384.209302       215
216.245098       204
193.596059       203
251.039474       190
188.053764       178
217.477169       165
203.153465       159
164.525510       138

```

```

201.114050      110
270.700000       85
187.206998       72
251.444744       57
244.090909       22
169.000000       20
144.842520       10
Name: count, dtype: int64

```

```
[ ]: d['Avg_Rating_City'].nunique()
```

```
[ ]: 6
```

```
[ ]: d['Avg_Rating_City'].value_counts()
```

```

[ ]: Avg_Rating_City
3.939031    15613
3.886635    11729
3.889413    11705
3.892180    11235
3.882244     6450
3.869203     688
Name: count, dtype: int64

```

```
[ ]: d['Avg_Rating_Cuisine'].nunique()
```

```
[ ]: 40
```

```
[ ]: d['Avg_Rating_Cuisine'].value_counts()
```

```

[ ]: Avg_Rating_Cuisine
3.928624    22738
3.963493     5503
3.818680     5271
3.862784     4174
3.873352     2786
3.875916     2573
3.926756     1466
3.913401     1351
3.882981     1221
3.807394     1070
3.825838       899
3.885698       814
4.047593       812
3.639863       697
3.608113       686
3.907037       516

```


3.757491	508
3.819056	485
4.069201	478
3.924747	379
3.756105	324
3.785305	298
3.944270	292
3.843777	251
3.861132	215
4.011132	204
3.676403	203
4.001790	190
3.962177	178
3.971023	165
3.685149	159
3.406122	138
3.634226	110
4.063223	85
3.883956	72
3.929711	57
4.100000	22
4.000000	20
3.835879	10
3.000000	1

Name: count, dtype: int64

```
[ ]: d['Avg_Rating_Restaurant'].nunique()
```

```
[ ]: 69
```

```
[ ]: d['Avg_Rating_Restaurant'].value_counts()
```

```
[ ]: Avg_Rating_Restaurant
```

4.000000	4673
4.100000	3789
3.950000	3522
4.150000	3332
3.911132	2576
...	
3.792857	68
3.975630	61
4.129334	56
4.402070	33
3.546130	22

Name: count, Length: 69, dtype: int64

```
[ ]: d['Best_Seller'].nunique()
```

```
[ ]: 14
```

```
[ ]: d['Best_Seller'].value_counts()
```

```
[ ]: Best_Seller
      NONE                45565
      BESTSELLER          7930
      MUST TRY            1777
      CHEF'S SPECIAL      695
      NOT ELIGIBLE FOR COUPONS 603
      SPICY                417
      NOT ON PRO           201
      NEW                 147
      SEASONAL             47
      VEGAN                17
      EGGLESS AVAILABLE    10
      GLUTEN FREE          6
      FODMAP FRIENDLY      5
      DAIRY FREE           1
      Name: count, dtype: int64
```

```
[ ]: d['Cuisine'].nunique()
```

```
[ ]: 39
```

```
[ ]: d['Cuisine'].value_counts()
```

```
[ ]: Cuisine
      Beverages      22738
      Desserts       5503
      Fast Food      5271
      Sichuan        4174
      Chinese        2786
      Biryani        2573
      North Indian   1466
      Shake          1351
      Street Food    1221
      Rolls          1070
      South Indian   899
      Pizza          814
      Seafood        812
      Mughlai        697
      Bbq            686
      Kebab          516
      Continental    508
      Momos          485
      Bakery         478
```

Ice Cream	379
Mandi	324
Burger	298
Hyderabadi	292
Coffee	251
Lucknowi	215
Healthy Food	204
Italian	203
Salad	190
Wraps	178
Mithai	165
Rajasthani	159
Shawarma	139
Sandwich	110
Mexican	85
Juices	72
American	57
Turkish	22
Tibetan	20
Kerala	10

Name: count, dtype: int64

```
[ ]: d['Delivery_Rating'].nunique()
```

```
[ ]: 16
```

```
[ ]: d['Delivery_Rating'].value_counts()
```

```
[ ]: Delivery_Rating
```

4.100000	12697
4.000000	8585
4.200000	7759
3.900000	6952
3.800000	5197
3.700000	4466
4.300000	3825
3.600000	3628
4.400000	1212
3.400000	934
3.500000	911
3.300000	468
4.500000	293
3.963184	273
3.200000	200
4.600000	21

Name: count, dtype: int64

```
[ ]: d['Delivery_Votes'].nunique()
```

```
[ ]: 138
```

```
[ ]: d['Delivery_Votes'].value_counts()
```

```
[ ]: Delivery_Votes
0      41251
183     700
819     309
940     287
520     268
...
478      19
198      14
148      10
18        9
217        5
Name: count, Length: 138, dtype: int64
```

```
[ ]: d['Dining_Rating'].nunique()
```

```
[ ]: 22
```

```
[ ]: d['Dining_Rating'].value_counts()
```

```
[ ]: Dining_Rating
3.822264    15967
3.800000     5475
4.100000     5191
4.000000     4610
4.200000     3933
4.300000     3527
3.900000     3199
3.600000     2222
3.700000     2070
3.500000     1661
4.400000     1459
3.400000     1342
3.100000     1292
3.300000     1234
3.000000     1042
3.200000      758
2.900000      597
2.800000      592
4.600000      434
4.500000      280
```

```
4.700000    279
2.700000    257
Name: count, dtype: int64
```

```
[ ]: d['Dining_Votes'].nunique()
```

```
[ ]: 165
```

```
[ ]: d['Dining_Votes'].value_counts()
```

```
[ ]: Dining_Votes
0      21984
848      844
16      736
32      607
4      588
...
367      34
27      32
482      23
62      22
18      16
Name: count, Length: 165, dtype: int64
```

```
[ ]: d['Is_Bestseller'].nunique()
```

```
[ ]: 1
```

```
[ ]: d['Is_Bestseller'].value_counts()
```

```
[ ]: Is_Bestseller
1      57421
Name: count, dtype: int64
```

```
[ ]: d['Is_Expensive'].nunique()
```

```
[ ]: 2
```

```
[ ]: d['Is_Expensive'].value_counts()
```

```
[ ]: Is_Expensive
0.0    41539
1.0    15881
Name: count, dtype: int64
```

```
[ ]: d['Is_Highly_Rated'].nunique()
```

```
[ ]: 2
```

```
[ ]: d['Is_Highly_Rated'].value_counts()
```

```
[ ]: Is_Highly_Rated
0.0    33864
1.0    23556
Name: count, dtype: int64
```

```
[ ]: d['Item_Name'].nunique()
```

```
[ ]: 30302
```

```
[ ]: d['Item_Name'].value_counts()
```

```
[ ]: Item_Name
Paneer Butter Masala          176
Veg Fried Rice                169
Chicken Fried Rice            160
Chicken 65                    126
Egg Fried Rice                117
...
Chicken Cheese Maggi [Serves 1] 1
Mayo Cheese Maggi              1
Peri Peri Cheese Maggi [Serves 1] 1
Schezwan Cheese Maggi [Serves 1] 1
Vanilla Hazelnut Cold Coffee    1
Name: count, Length: 30302, dtype: int64
```

```
[ ]: d['Log_Price'].nunique()
```

```
[ ]: 1780
```

```
[ ]: d['Log_Price'].value_counts()
```

```
[ ]: Log_Price
5.198497    1032
4.795791    1009
5.303305     972
5.398163     964
5.017280     940
...
7.123673      1
6.918695      1
6.046615      1
5.599458      1
4.589650      1
```

```
Name: count, Length: 1780, dtype: int64
```

```
[ ]: d['Place_Name'].nunique()
```

```
[ ]: 144
```

```
[ ]: d['Place_Name'].value_counts()
```

```
[ ]: Place_Name
     C Scheme          2287
     Chembur          1826
     Charminar        1751
     Abids            1668
     Malakpet         1465
     ...
     East Street        25
     Crystal Palm Mall  24
     Kora Food Street   20
     Katraj             16
     Deccan Gymkhana    4
     Name: count, Length: 144, dtype: int64
```

```
[ ]: d['Price_per_Vote'].nunique()
```

```
[ ]: 7945
```

```
[ ]: d['Price_per_Vote'].value_counts()
```

```
[ ]: Price_per_Vote
     180.000000    554
     220.000000    545
     150.000000    539
     120.000000    538
     200.000000    516
     ...
     48.900000     1
     34.666667     1
     36.111111     1
     0.287879      1
     0.259366      1
     Name: count, Length: 7945, dtype: int64
```

```
[ ]: d['Prices'].nunique()
```

```
[ ]: 1780
```

```
[ ]: d['Prices'].value_counts()
```

```
[ ]: Prices
180.00    1032
120.00    1009
200.00     972
220.00     964
150.00     940
...
1240.00     1
1010.00     1
421.68      1
269.28      1
97.46       1
Name: count, Length: 1780, dtype: int64
```

```
[ ]: d['Restaurant_Name'].nunique()
```

```
[ ]: 410
```

```
[ ]: d['Restaurant_Name'].value_counts()
```

```
[ ]: Restaurant_Name
The Momoz Hub          700
Savoury Sea Shell     595
Raj Restaurant         555
Capital Multi Cuisine Restaurant  545
Casa Blanca           508
...
Mohammedia Shawarma     9
Arabian Food Court      5
Akbar Fast Food         5
Al - Hamoor Arabian Restaurant  5
Sukanta                 4
Name: count, Length: 410, dtype: int64
```

```
[ ]: d['Restaurant_Popularity'].nunique()
```

```
[ ]: 230
```

```
[ ]: d['Restaurant_Popularity'].value_counts()
```

```
[ ]: Restaurant_Popularity
406    812
188    752
247    741
700    700
460    692
...
```



```

5      15
14     14
13     13
9       9
4       4
Name: count, Length: 230, dtype: int64

```

```
[ ]: d['Total_Votes'].nunique()
```

```
[ ]: 258
```

```
[ ]: d['Total_Votes'].value_counts()
```

```

[ ]: Total_Votes
0      13332
848     844
183     700
16     595
668     575
...
808      22
478      19
18       16
199      14
217       5
Name: count, Length: 258, dtype: int64

```

```
[ ]: d['Votes'].nunique()
```

```
[ ]: 619
```

```
[ ]: d['Votes'].value_counts
```

```

[ ]: Votes
0      33638
5      1895
6      1587
7      1217
8      1078
...
1259      1
404       1
336       1
848       1
300       1
Name: count, Length: 619, dtype: int64

```

1.CATEGORY: Identifying the Next Winning Cuisine Problem

Find which cuisine type has:

High ratings

Strong demand (votes)

But low competition (few restaurants)

Graphs to Use 1. Scatter plot (Average_Rating vs. Total_Votes, color = Cuisine, size = restaurant count)

2. Bar chart (Cuisine vs. Count of Restaurants)

3. Box plot (Cuisine vs. Avg_Price_Cuisine)

4. Heatmap of Avg_Rating_Cuisine and Avg_Price_Cuisine

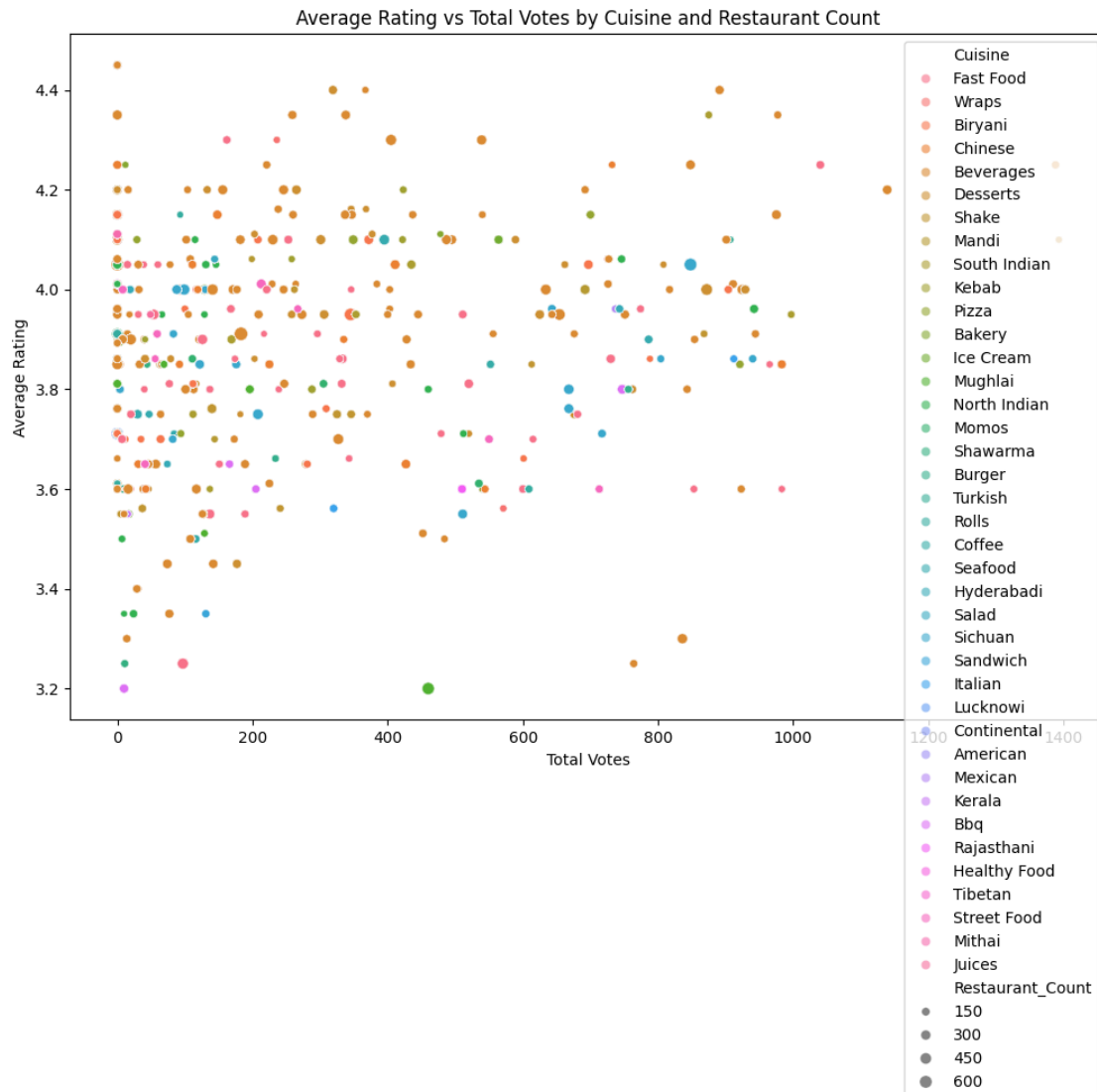
5. Bubble chart combining votes, ratings, and price

```
[ ]: # Create a new column 'Restaurant_Count' which contains the count of
      ↪ occurrences of each restaurant name
d['Restaurant_Count'] = d.groupby('Restaurant_Name')['Restaurant_Name'].
      ↪ transform('count')

# Create a scatter plot of 'Average_Rating' versus 'Total_Votes',
# coloring the points by 'Cuisine', and sizing them by 'Restaurant_Count'
plt.figure(figsize=(12, 8))
sns.scatterplot(data=d, x='Total_Votes', y='Average_Rating', hue='Cuisine',
      ↪ size='Restaurant_Count', alpha=0.6)

# Add titles and labels for clarity
plt.title('Average Rating vs Total Votes by Cuisine and Restaurant Count')
plt.xlabel('Total Votes')
plt.ylabel('Average Rating')

# Display the plot
plt.show()
```



```
[ ]: # Count the number of restaurants for each cuisine
cuisine_counts = d['Cuisine'].value_counts().reset_index()
cuisine_counts.columns = ['Cuisine', 'Restaurant_Count']

# Create a bar chart
plt.figure(figsize=(15, 7))
sns.barplot(data=cuisine_counts, x='Cuisine', y='Restaurant_Count',
            palette='viridis') # Changed palette to 'viridis'

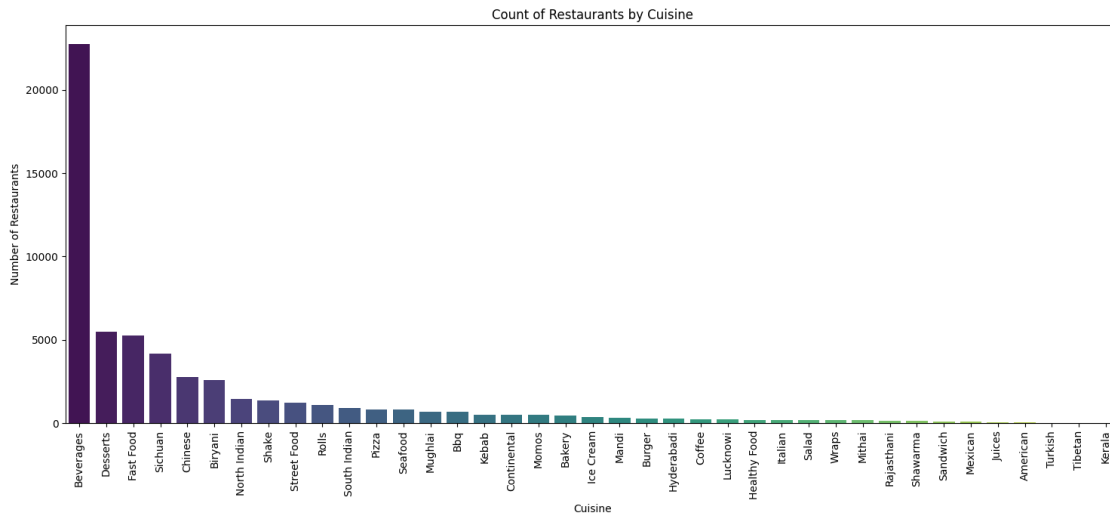
# Rotate x-axis labels for better readability
plt.xticks(rotation=90)

# Add titles and labels
```

```
plt.title('Count of Restaurants by Cuisine')
plt.xlabel('Cuisine')
plt.ylabel('Number of Restaurants')

# Adjust layout to prevent labels from overlapping
plt.tight_layout()

# Display the plot
plt.show()
```



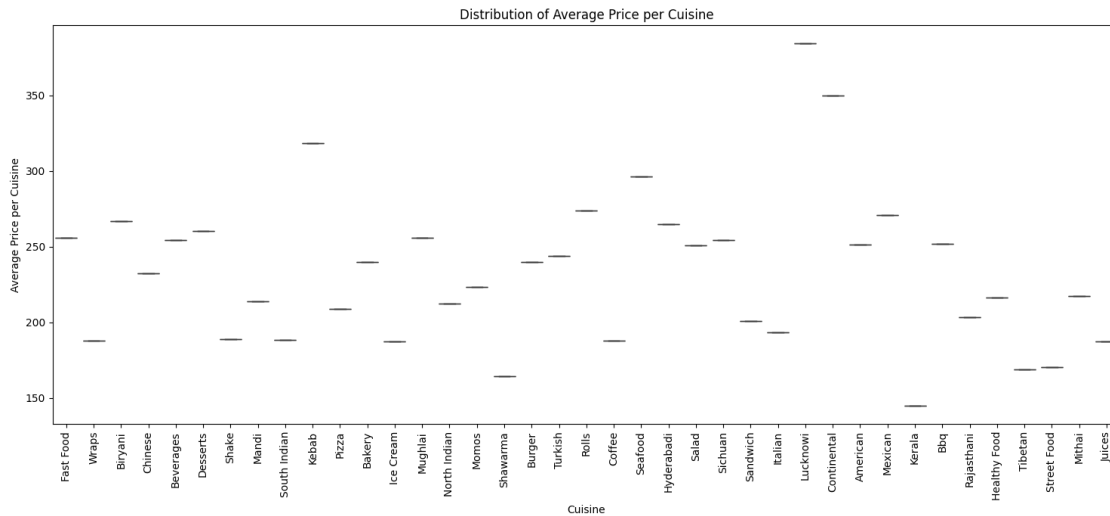
```
[ ]: # Create a box plot of 'Avg_Price_Cuisine' for each cuisine
plt.figure(figsize=(15, 7))
sns.boxplot(data=d, x='Cuisine', y='Avg_Price_Cuisine')

# Rotate x-axis labels for better readability
plt.xticks(rotation=90)

# Add titles and labels
plt.title('Distribution of Average Price per Cuisine')
plt.xlabel('Cuisine')
plt.ylabel('Average Price per Cuisine')

# Adjust layout to prevent labels from overlapping
plt.tight_layout()

# Display the plot
plt.show()
```



```
[ ]: # Calculate the average rating and average price for each cuisine
cuisine_avg_metrics = d.groupby('Cuisine')[['Avg_Rating_Cuisine', 'Avg_Price_Cuisine']].mean().reset_index()

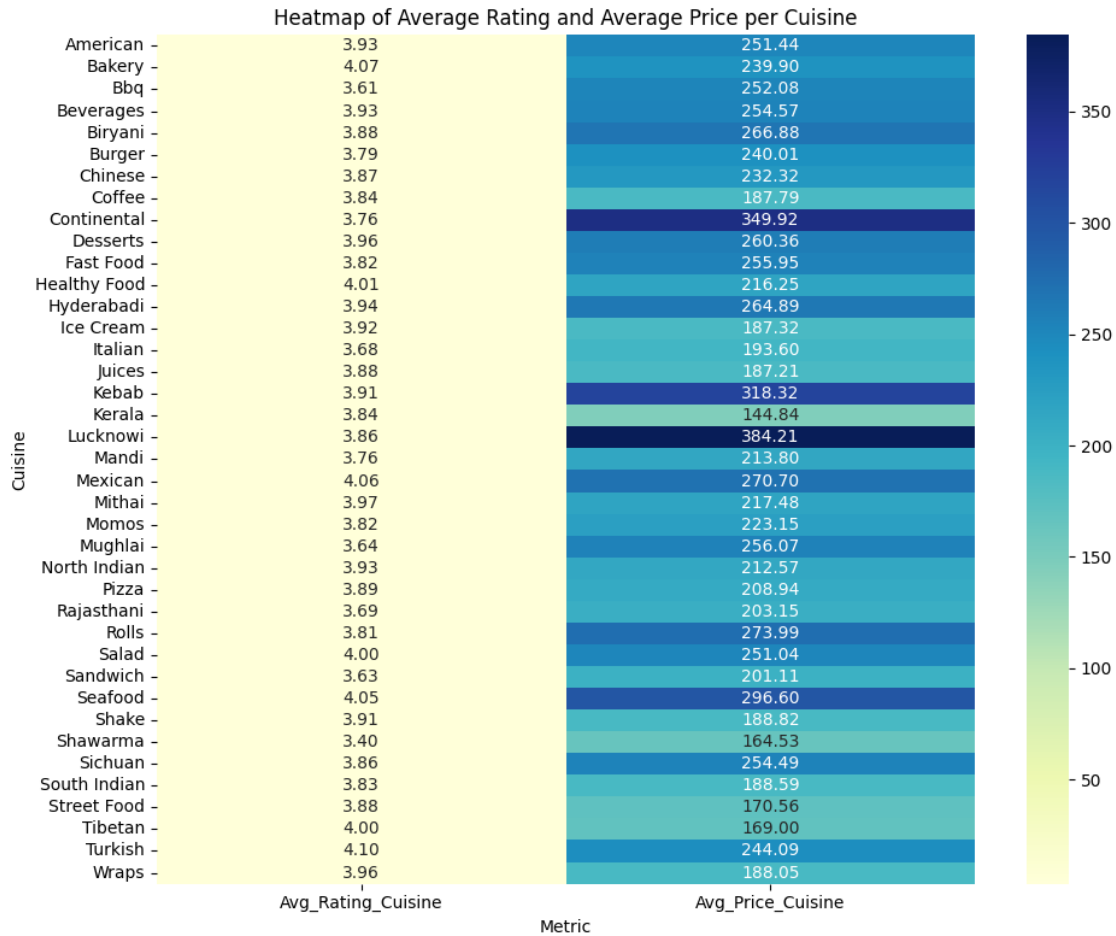
# Create a pivot table for the heatmap
heatmap_data = cuisine_avg_metrics.set_index('Cuisine')

# Create a heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(heatmap_data, annot=True, cmap='YlGnBu', fmt=".2f")

# Add titles and labels
plt.title('Heatmap of Average Rating and Average Price per Cuisine')
plt.xlabel('Metric')
plt.ylabel('Cuisine')

# Adjust layout
plt.tight_layout()

# Display the plot
plt.show()
```



```
[ ]: # Aggregate data by Cuisine to get average rating, average price, and sum of
      ↳total votes
cuisine_bubble_data = d.groupby('Cuisine').agg(
    Avg_Rating_Cuisine=('Average_Rating', 'mean'),
    Avg_Price_Cuisine=('Prices', 'mean'),
    Total_Votes=('Total_Votes', 'sum')
).reset_index()

# Create a bubble chart
plt.figure(figsize=(14, 8))
sns.scatterplot(data=cuisine_bubble_data, x='Avg_Price_Cuisine',
    ↳y='Avg_Rating_Cuisine', size='Total_Votes', hue='Cuisine', alpha=0.7,
    ↳sizes=(100, 2000))

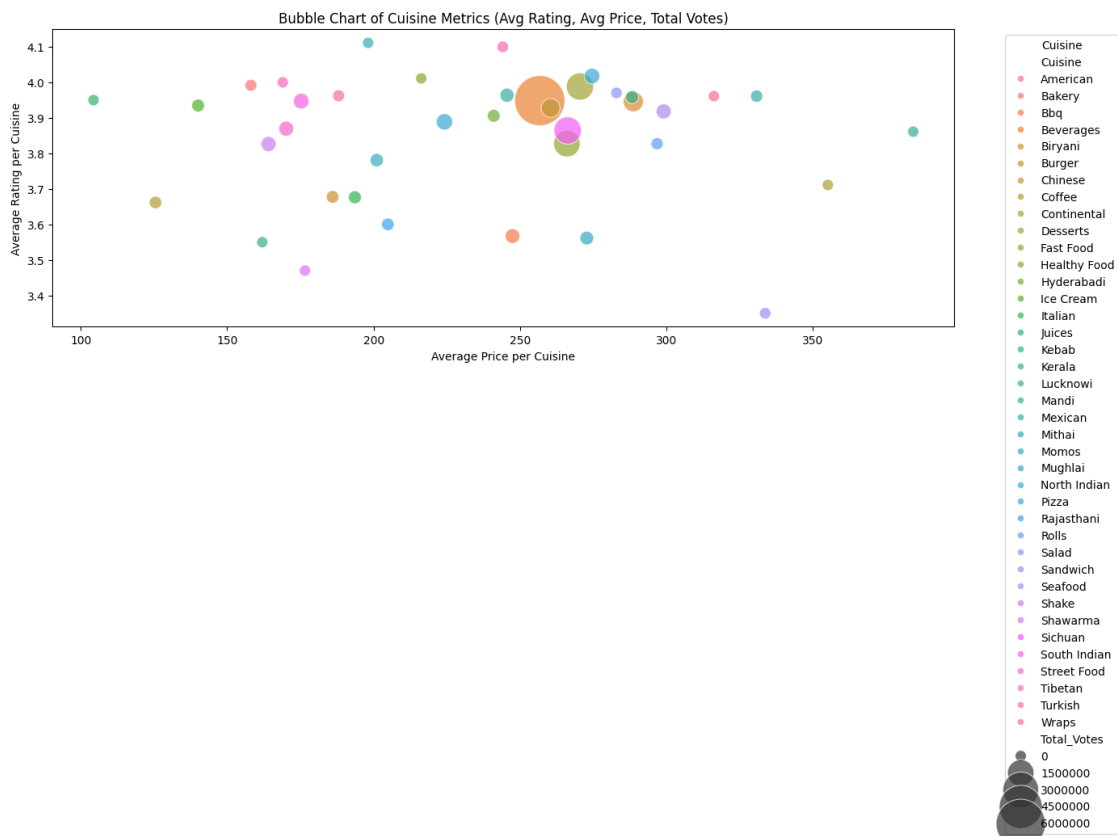
# Add titles and labels
plt.title('Bubble Chart of Cuisine Metrics (Avg Rating, Avg Price, Total
    ↳Votes)')
```

```
plt.xlabel('Average Price per Cuisine')
plt.ylabel('Average Rating per Cuisine')

# Add legend for bubble size
plt.legend(title='Cuisine', bbox_to_anchor=(1.05, 1), loc='upper left')

# Adjust layout
plt.tight_layout()

# Display the plot
plt.show()
```



Insights to Aim For

1. High-rating
2. High-demand cuisines with low saturation → ideal for market entry.

Example: “Italian and Thai cuisines have top ratings but fewer total restaurants.”

2.MARKET: Pinpointing the Best City / Location Problem

Find which city or place offers the best potential for growth:

1. High consumer engagement (votes)

2. High average rating
3. Low number of established competitors (Is_Highly_Rated restaurants)

Graphs to Use

1. Bar chart (City vs. Avg_Rating_City)
2. Scatter plot (Avg_Rating_City vs. Total_Votes)
3. Stacked bar chart (City vs. Count of Is_Highly_Rated and total restaurants)
4. Heatmap or Bubble plot (Place_Name vs. Restaurant_Popularity)
5. Line/Bar chart comparing Avg_Price_City and Avg_Rating_City

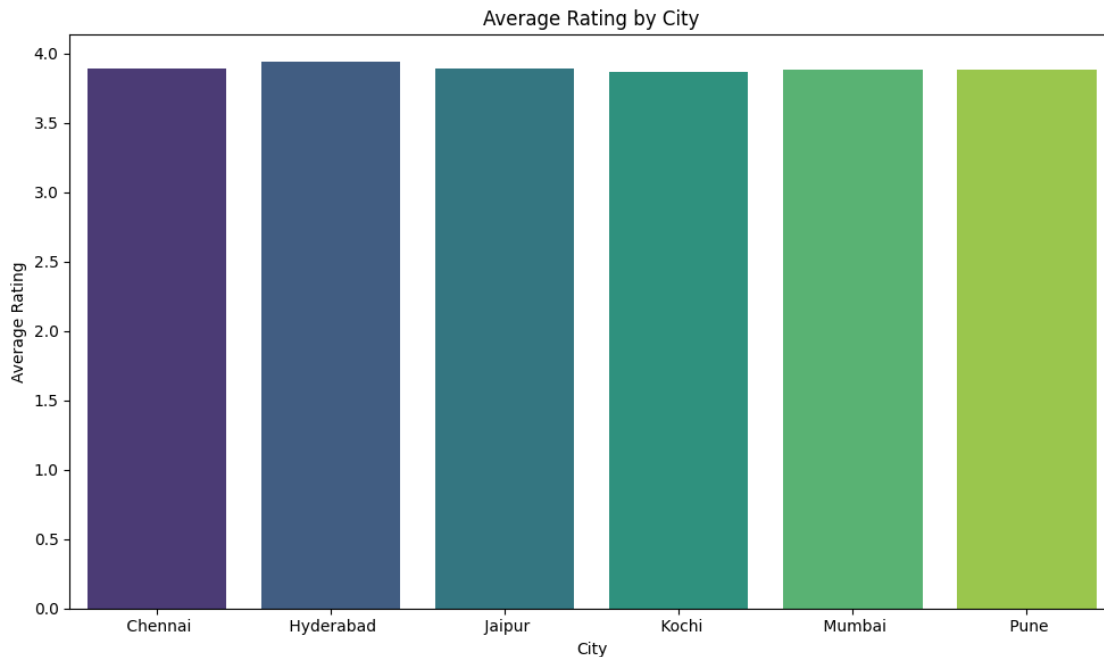
```
[ ]: # Calculate the average rating for each city
city_avg_rating = d.groupby('City')['Avg_Rating_City'].mean().reset_index()

# Create a bar chart
plt.figure(figsize=(10, 6))
sns.barplot(data=city_avg_rating, x='City', y='Avg_Rating_City',
            palette='viridis') # Changed palette to 'viridis' as a suitable
                               # violet-to-green option

# Add titles and labels
plt.title('Average Rating by City')
plt.xlabel('City')
plt.ylabel('Average Rating')

# Adjust layout
plt.tight_layout()

# Display the plot
plt.show()
```

```
[ ]: # Aggregate data by City to get average rating and sum of total votes
city_metrics = d.groupby('City').agg(
    Avg_Rating_City=('Avg_Rating_City', 'mean'),
    Total_Votes=('Total_Votes', 'sum')
).reset_index()

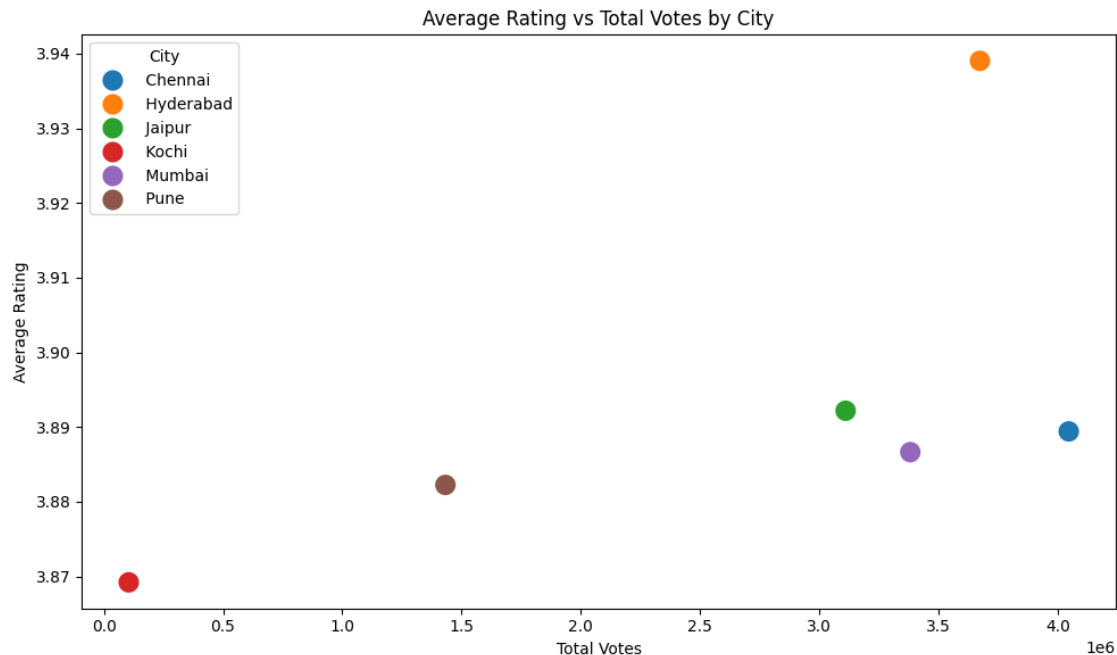
# Create a scatter plot
plt.figure(figsize=(10, 6))
sns.scatterplot(data=city_metrics, x='Total_Votes', y='Avg_Rating_City',
                hue='City', s=200) # Increased size of points for better visibility

# Add titles and labels
plt.title('Average Rating vs Total Votes by City')
plt.xlabel('Total Votes')
plt.ylabel('Average Rating')

# Add legend
plt.legend(title='City')

# Adjust layout
plt.tight_layout()

# Display the plot
plt.show()
```



```
[ ]: # Aggregate data by City to get the total count and the count of highly-rated
    ↪restaurants
city_highlyRated = d.groupby('City').agg(
    Total_Restaurants=('Restaurant_Name', 'count'),
    Highly_Rated_Restaurants=('Is_Highly_Rated', lambda x: (x == 1).sum())
).reset_index()

# Create a stacked bar chart
plt.figure(figsize=(12, 7))

# Plot the total restaurants
sns.barplot(data=city_highlyRated, x='City', y='Total_Restaurants',
    ↪color='darkgreen', label='Total Restaurants') # Changed color to darkgreen

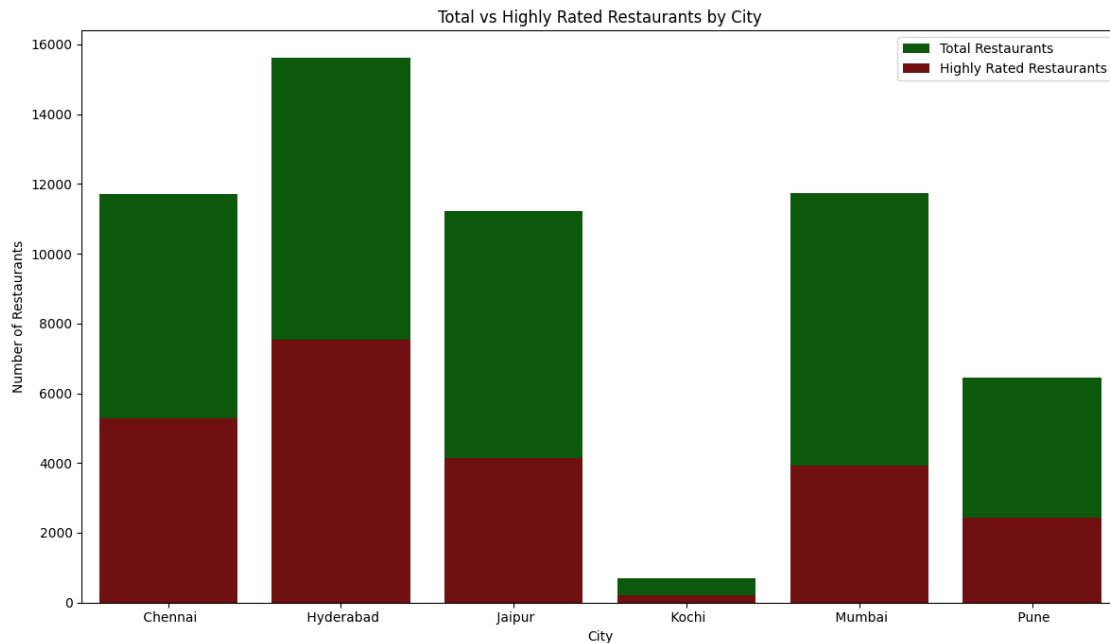
# Plot the highly-rated restaurants on top
sns.barplot(data=city_highlyRated, x='City', y='Highly_Rated_Restaurants',
    ↪color='maroon', label='Highly Rated Restaurants') # Changed color to maroon

# Add titles and labels
plt.title('Total vs Highly Rated Restaurants by City')
plt.xlabel('City')
plt.ylabel('Number of Restaurants')

# Add legend
plt.legend()
```

```
# Adjust layout
plt.tight_layout()

# Display the plot
plt.show()
```



```
[ ]: # Calculate the average restaurant popularity for each place name
place_popularity = d.groupby('Place_Name')['Restaurant_Popularity'].mean().
    ↪reset_index()

# Create a pivot table for the heatmap
heatmap_data_place = place_popularity.set_index('Place_Name')

# Create a heatmap
plt.figure(figsize=(12, 30)) # Increased the height of the figure

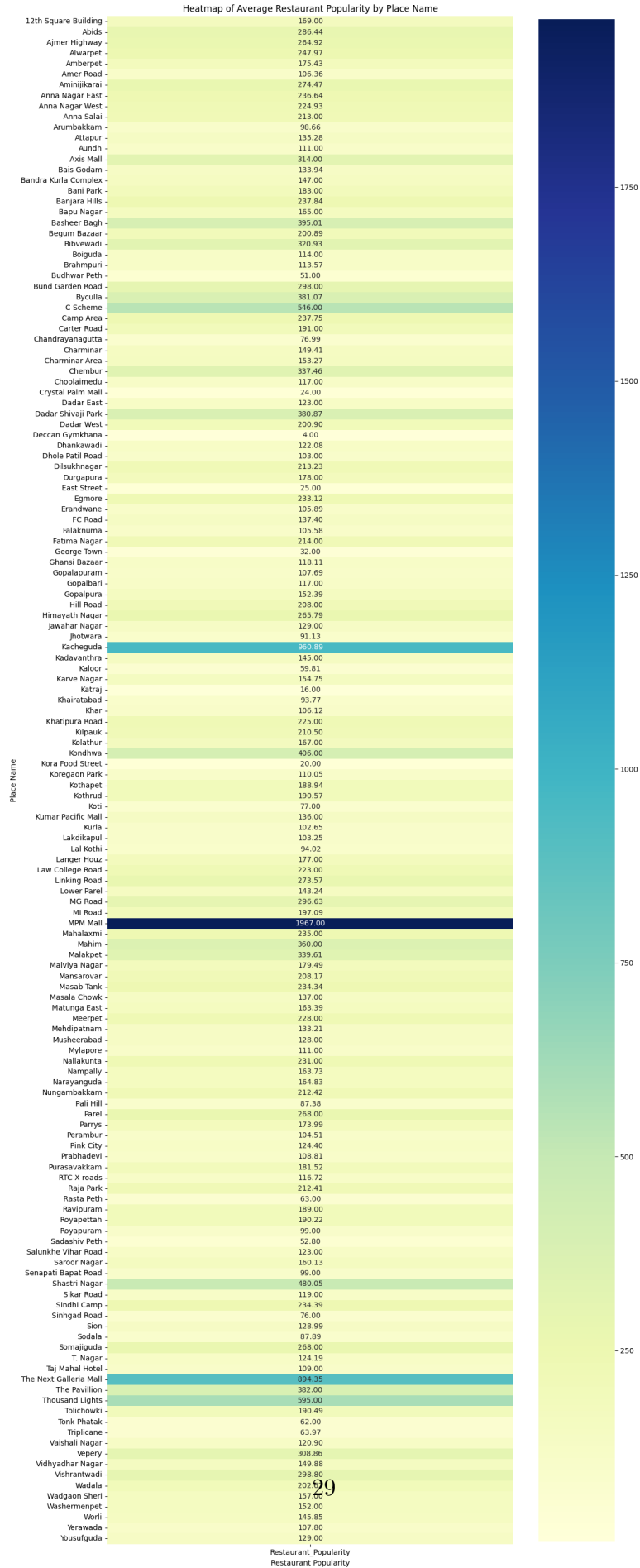
# Create a heatmap
sns.heatmap(heatmap_data_place, annot=True, cmap='YlGnBu', fmt=".2f")

# Add titles and labels
plt.title('Heatmap of Average Restaurant Popularity by Place Name')
plt.xlabel('Restaurant Popularity')
plt.ylabel('Place Name')

# Adjust layout
```

```
plt.tight_layout()

# Display the plot
plt.show()
```



```
[ ]: # Aggregate data by City to get average price and average rating
city_comparison_data = d.groupby('City').agg(
    Avg_Price_City=('Avg_Price_City', 'mean'),
    Avg_Rating_City=('Avg_Rating_City', 'mean')
).reset_index()

# Create a figure and a set of subplots with a secondary y-axis
fig, ax1 = plt.subplots(figsize=(12, 7))
ax2 = ax1.twinx()

# Create a bar chart for Avg_Price_City
sns.barplot(data=city_comparison_data, x='City', y='Avg_Price_City', ax=ax1,
    color='gold', label='Average Price') # Changed color to gold

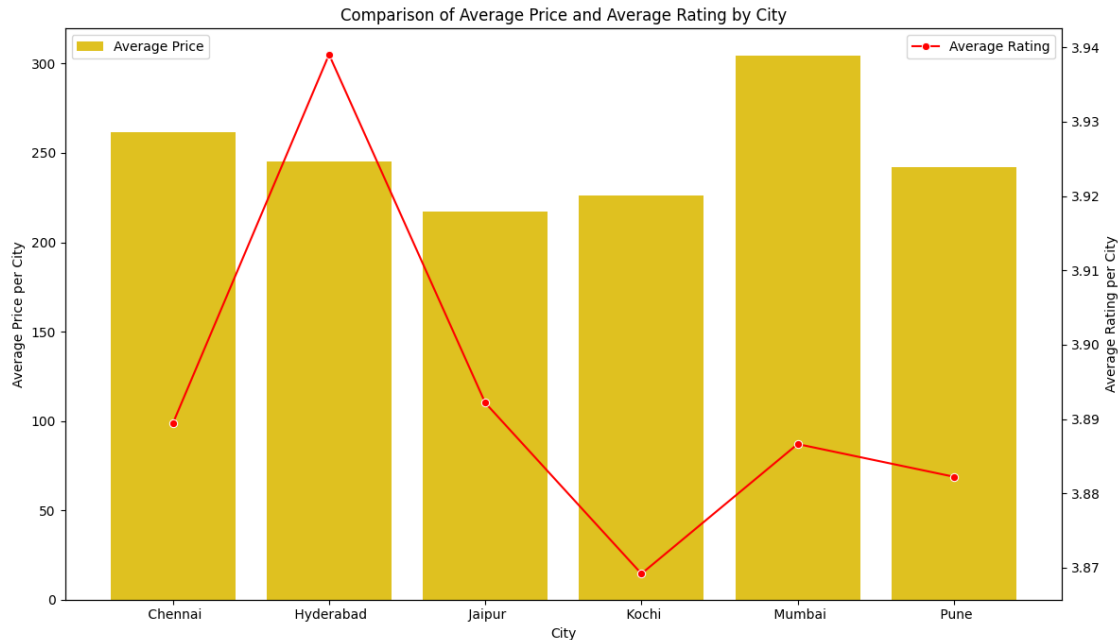
# Create a line chart for Avg_Rating_City on the secondary y-axis
sns.lineplot(data=city_comparison_data, x='City', y='Avg_Rating_City', ax=ax2,
    color='red', marker='o', label='Average Rating')

# Add titles and labels
ax1.set_title('Comparison of Average Price and Average Rating by City')
ax1.set_xlabel('City')
ax1.set_ylabel('Average Price per City')
ax2.set_ylabel('Average Rating per City')

# Add legends
ax1.legend(loc='upper left')
ax2.legend(loc='upper right')

# Adjust layout
fig.tight_layout()

# Display the plot
plt.show()
```



Insights to Aim For

1. “Which city has high demand but fewer top restaurants?”
2. “Which neighborhood shows strong popularity with limited supply?”

3.CHANNEL: Defining the Service Model (Delivery vs. Dining Focus) Problem

Determine whether to focus on delivery or dining:

1. Compare performance across Dining_Rating vs. Delivery_Rating
2. Measure demand using Dining_Votes vs. Delivery_Votes
3. Identify which dominates for your target cuisine/city

Graphs to Use 1. Scatter plot (Dining_Rating vs. Delivery_Rating) 2. Box plot (Cuisine vs. Dining_Votes / Delivery_Votes) 3. Heatmap (Dining_Rating, Delivery_Rating, Dining_Votes, Delivery_Votes) 4. Stacked bar chart (Cuisine vs. average ratings/votes split) 5. Scatter plot with segmentation (color = Is_Bestseller)

```
[ ]: # Create a scatter plot of Dining_Rating vs. Delivery_Rating
plt.figure(figsize=(8, 6))
sns.scatterplot(data=d, x='Dining_Rating', y='Delivery_Rating', alpha=0.6)

# Add titles and labels
plt.title('Dining Rating vs Delivery Rating')
plt.xlabel('Dining Rating')
plt.ylabel('Delivery Rating')
```

```
# Display the plot
plt.show()
```



```
[ ]: # Create a box plot of 'Dining_Votes' for each cuisine
plt.figure(figsize=(15, 7))
sns.boxplot(data=d, x='Cuisine', y='Dining_Votes')

# Rotate x-axis labels for better readability
plt.xticks(rotation=90)

# Add titles and labels
plt.title('Distribution of Dining Votes by Cuisine')
plt.xlabel('Cuisine')
plt.ylabel('Dining Votes')

# Adjust layout
plt.tight_layout()

# Display the plot
```



```
plt.show()

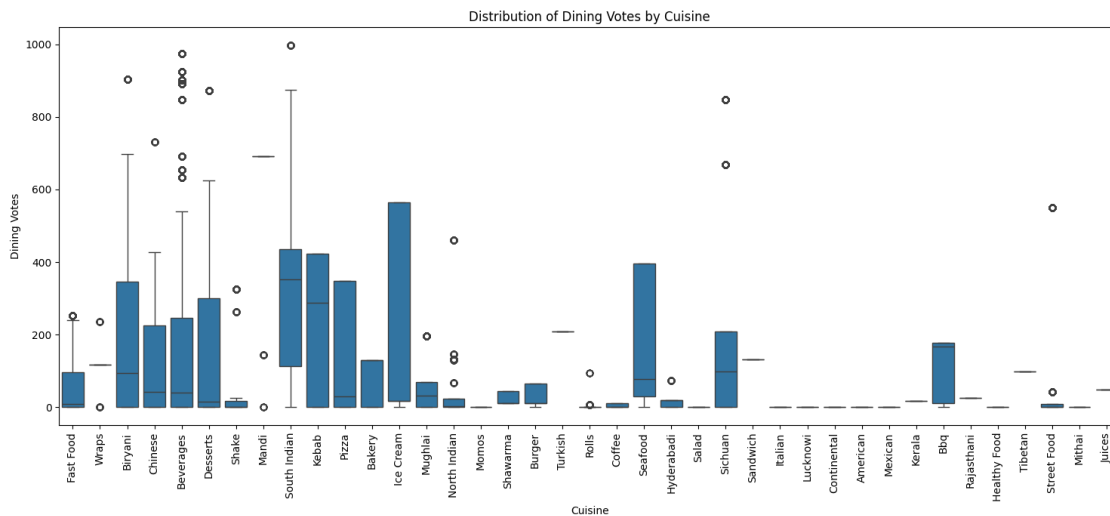
# Create a box plot of 'Delivery_Votes' for each cuisine
plt.figure(figsize=(15, 7))
sns.boxplot(data=d, x='Cuisine', y='Delivery_Votes')

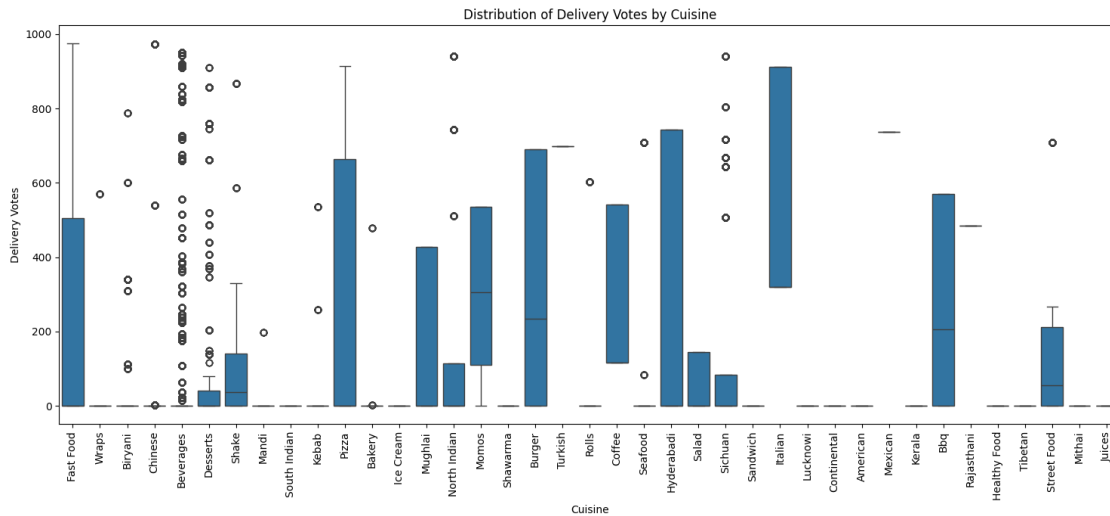
# Rotate x-axis labels for better readability
plt.xticks(rotation=90)

# Add titles and labels
plt.title('Distribution of Delivery Votes by Cuisine')
plt.xlabel('Cuisine')
plt.ylabel('Delivery Votes')

# Adjust layout
plt.tight_layout()

# Display the plot
plt.show()
```





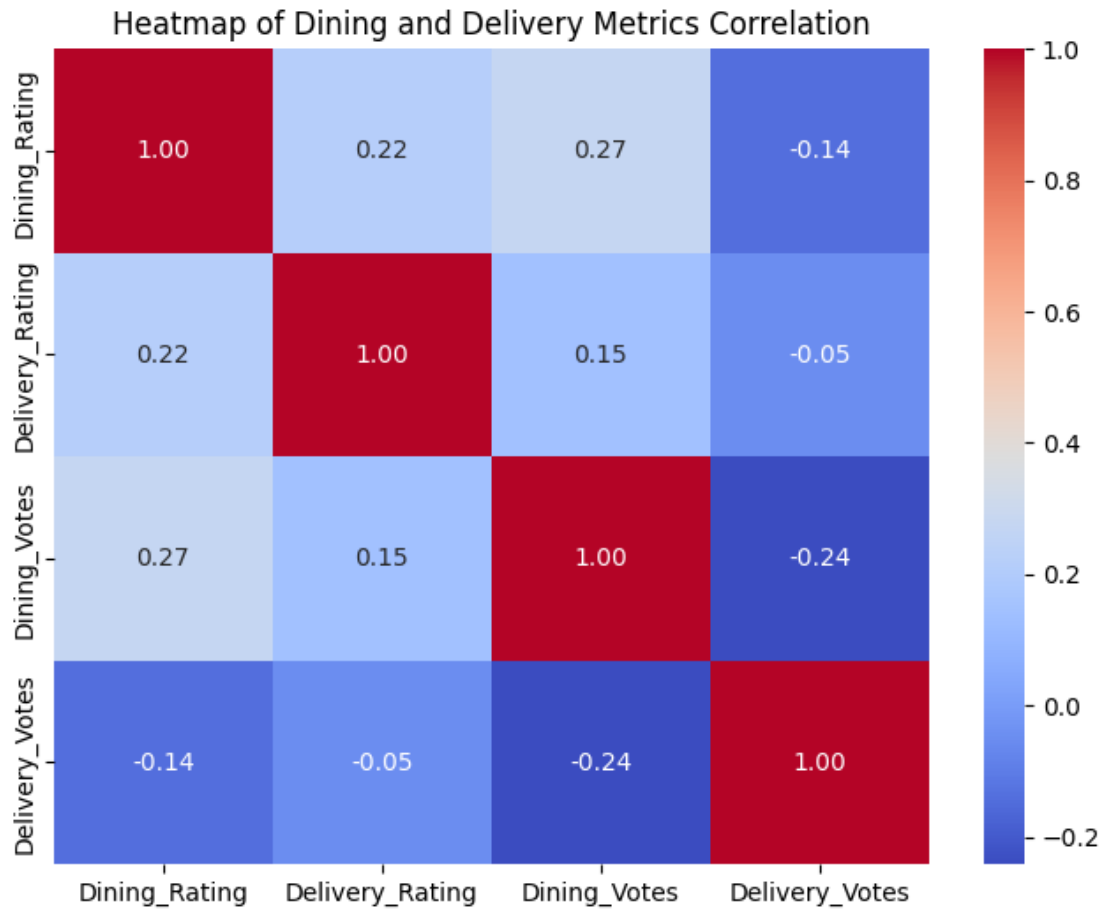
```
[ ]: # Select the relevant columns
heatmap_data_channel = d[['Dining_Rating', 'Delivery_Rating', 'Dining_Votes',
↵ 'Delivery_Votes']]

# Calculate the correlation matrix
correlation_matrix = heatmap_data_channel.corr()

# Create a heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f")

# Add titles and labels
plt.title('Heatmap of Dining and Delivery Metrics Correlation')

# Display the plot
plt.show()
```



```
[ ]: # Aggregate data by Cuisine to get average dining and delivery ratings and votes
cuisine_channel_metrics = d.groupby('Cuisine').agg(
    Avg_Dining_Rating=('Dining_Rating', 'mean'),
    Avg_Delivery_Rating=('Delivery_Rating', 'mean'),
    Avg_Dining_Votes=('Dining_Votes', 'mean'),
    Avg_Delivery_Votes=('Delivery_Votes', 'mean')
).reset_index()

# Create stacked bar charts for ratings and votes separately
fig, axes = plt.subplots(2, 1, figsize=(15, 14))

# Stacked bar chart for average ratings
cuisine_channel_metrics[['Cuisine', 'Avg_Dining_Rating',
    ↪ 'Avg_Delivery_Rating']].set_index('Cuisine').plot(kind='bar', stacked=True,
    ↪ ax=axes[0])
axes[0].set_title('Average Dining vs. Delivery Ratings by Cuisine')
axes[0].set_xlabel('Cuisine')
axes[0].set_ylabel('Average Rating')
```

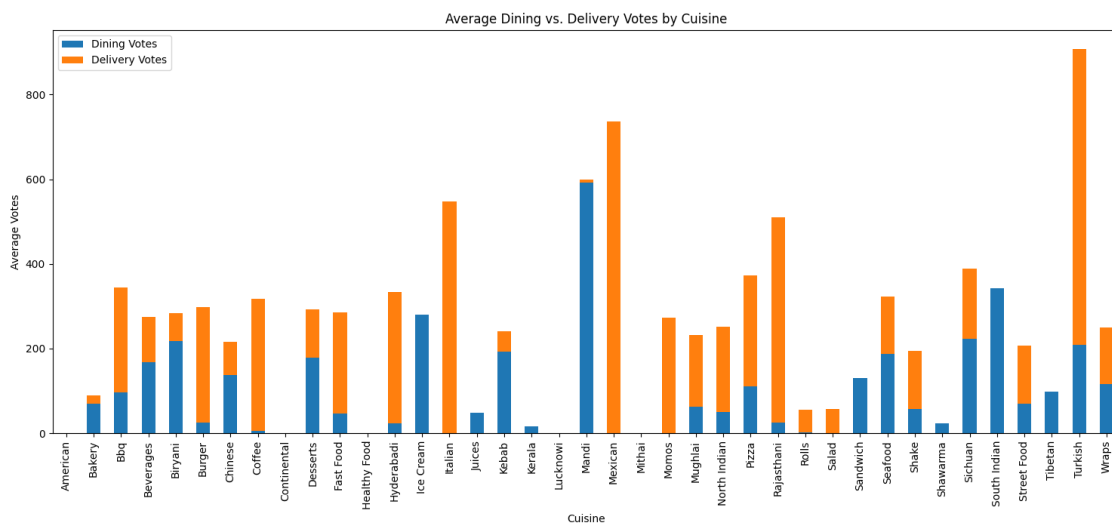
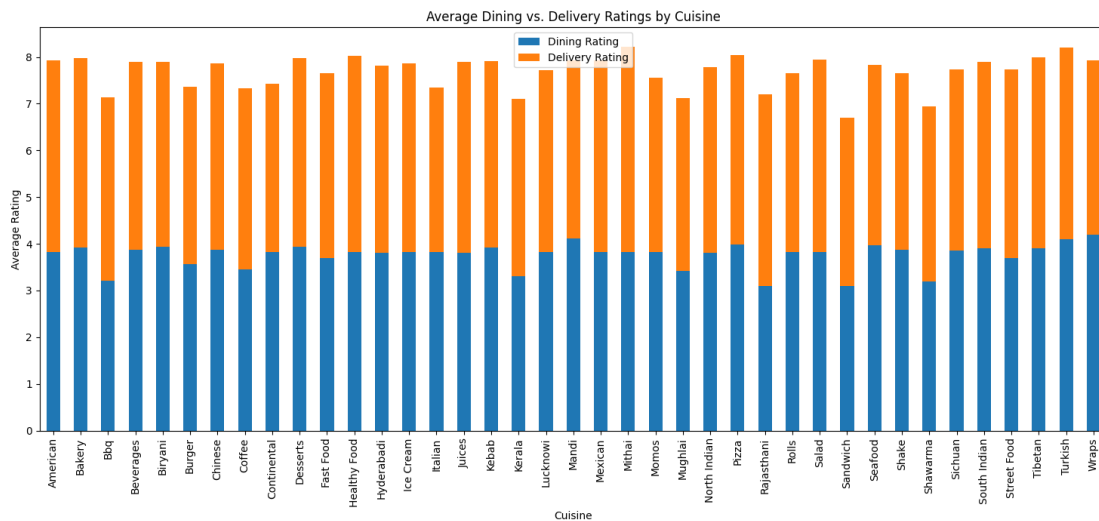
```

axes[0].tick_params(axis='x', rotation=90)
axes[0].legend(['Dining Rating', 'Delivery Rating'])

# Stacked bar chart for average votes
cuisine_channel_metrics[['Cuisine', 'Avg_Dining_Votes', 'Avg_Delivery_Votes']].
    .set_index('Cuisine').plot(kind='bar', stacked=True, ax=axes[1])
axes[1].set_title('Average Dining vs. Delivery Votes by Cuisine')
axes[1].set_xlabel('Cuisine')
axes[1].set_ylabel('Average Votes')
axes[1].tick_params(axis='x', rotation=90)
axes[1].legend(['Dining Votes', 'Delivery Votes'])

plt.tight_layout()
plt.show()

```



```
[ ]: # Create a scatter plot of 'Prices' versus 'Average_Rating',
# coloring the points based on 'Is_Bestseller'
plt.figure(figsize=(10, 6))
sns.scatterplot(data=d, x='Prices', y='Average_Rating', hue='Is_Bestseller',
               alpha=0.6, palette='viridis') # Added palette='viridis'

# Add titles and labels
plt.title('Average Rating vs Prices by Bestseller Status')
plt.xlabel('Prices')
plt.ylabel('Average Rating')

# Add legend
plt.legend(title='Is Bestseller')

# Display the plot
plt.show()
```



Insights to Aim For

1. Delivery-first strategy if delivery metrics outperform dining across cuisines.
2. Dining-first if dine-in experiences lead to higher ratings or engagement.

STATISTICAL ANALYSIS

To strengthen your findings (and score higher on the 20% “Statistical Analysis”):

Correlation coefficients: Between Dining_Rating & Delivery_Rating, and Dining_Votes & Delivery_Votes

Z-scores or percentiles: Identify outlier cuisines/cities

Hypothesis testing:

H : No difference between average Dining_Rating and Delivery_Rating

H : There is a significant difference

Cluster analysis: Group cuisines or cities based on performance metrics

```
[ ]: # Calculate the correlation between Dining_Rating and Delivery_Rating
correlation_ratings = d['Dining_Rating'].corr(d['Delivery_Rating'])
print(f"Correlation between Dining_Rating and Delivery_Rating: {correlation_ratings:.2f}")

# Calculate the correlation between Dining_Votes and Delivery_Votes
correlation_votes = d['Dining_Votes'].corr(d['Delivery_Votes'])
print(f"Correlation between Dining_Votes and Delivery_Votes: {correlation_votes:.2f}")
```

Correlation between Dining_Rating and Delivery_Rating: 0.22

Correlation between Dining_Votes and Delivery_Votes: -0.24

4 Task

Identify outlier cuisines and cities based on a numerical metric using Z-scores or percentiles.

4.1 Choose a metric

4.1.1 Subtask:

Select a numerical metric (e.g., average rating, total votes, average price) to analyze for outliers in cuisines and cities.

4.2 Aggregate data

4.2.1 Subtask:

Aggregate the data by 'Cuisine' and 'City' and calculate the mean of the 'Average_Rating' for each group.

Reasoning: Aggregate the data by 'Cuisine' and 'City' and calculate the mean of the 'Average_Rating' for each group as requested by the instructions.

```
[ ]: cuisine_avg_rating = d.groupby('Cuisine')['Average_Rating'].mean().reset_index()
city_avg_rating = d.groupby('City')['Average_Rating'].mean().reset_index()
display(cuisine_avg_rating.head())
display(city_avg_rating.head())
```

	Cuisine	Average_Rating
0	Bakery	3.992362
1	Beverages	3.987286
2	Biryani	3.920602
3	Burger	3.600000
4	Chinese	3.889904

	City	Average_Rating
0	Hyderabad	3.936763

Reasoning: The previous command failed because the dataframe 'd' was not defined in the current session. Need to re-execute the cell where 'd' was defined.

```
[ ]: zomato=pd.read_csv("Zomato Dataset.csv")
d=zomato
cuisine_avg_rating = d.groupby('Cuisine')['Average_Rating'].mean().reset_index()
city_avg_rating = d.groupby('City')['Average_Rating'].mean().reset_index()
display(cuisine_avg_rating.head())
display(city_avg_rating.head())
```

	Cuisine	Average_Rating
0	Bakery	3.992362
1	Beverages	3.987286
2	Biryani	3.920602
3	Burger	3.600000
4	Chinese	3.889904

	City	Average_Rating
0	Hyderabad	3.936763

Reasoning: The previous command failed because the pandas library was not imported. Need to re-execute the cell where pandas was imported.

```
[ ]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')
zomato=pd.read_csv("Zomato Dataset.csv")
d=zomato
cuisine_avg_rating = d.groupby('Cuisine')['Average_Rating'].mean().reset_index()
city_avg_rating = d.groupby('City')['Average_Rating'].mean().reset_index()
display(cuisine_avg_rating.head())
display(city_avg_rating.head())
```

	Cuisine	Average_Rating
0	American	3.961132
1	Bakery	3.992362
2	Bbq	3.650000

```
3 Beverages      3.992966
4   Biryani      3.905967
```

```
      City Average_Rating
0   Chennai      3.895680
1  Hyderabad      3.939031
2    Mumbai      3.914389
```

```
<google.colab._quickchart_helpers.SectionTitle at 0x7d35a83b85c0>
```

```
from matplotlib import pyplot as plt
_df_0['Average_Rating'].plot(kind='hist', bins=20, title='Average_Rating')
plt.gca().spines[['top', 'right']].set_visible(False)
```

```
<google.colab._quickchart_helpers.SectionTitle at 0x7d35789c4170>
```

```
from matplotlib import pyplot as plt
import seaborn as sns
_df_1.groupby('City').size().plot(kind='barh', color=sns.palettes.
    ↪mpl_palette('Dark2'))
plt.gca().spines[['top', 'right']].set_visible(False)
```

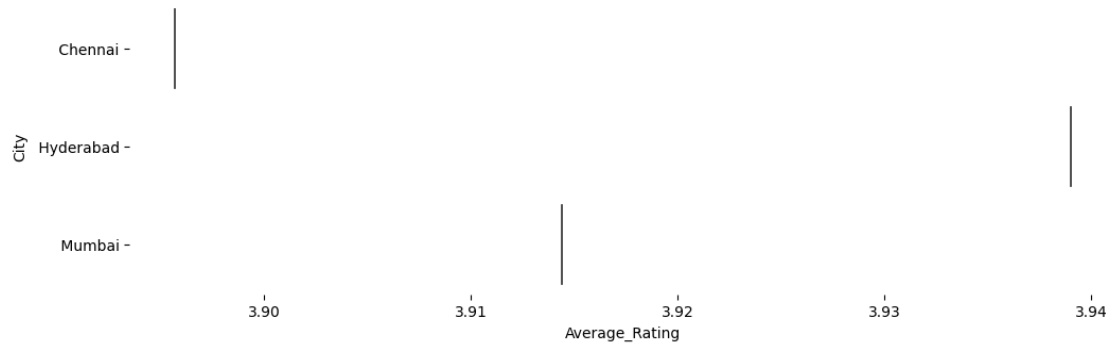
```
<google.colab._quickchart_helpers.SectionTitle at 0x7d3578a87bf0>
```

```
from matplotlib import pyplot as plt
_df_2['Average_Rating'].plot(kind='line', figsize=(8, 4), title='Average_Rating')
plt.gca().spines[['top', 'right']].set_visible(False)
```

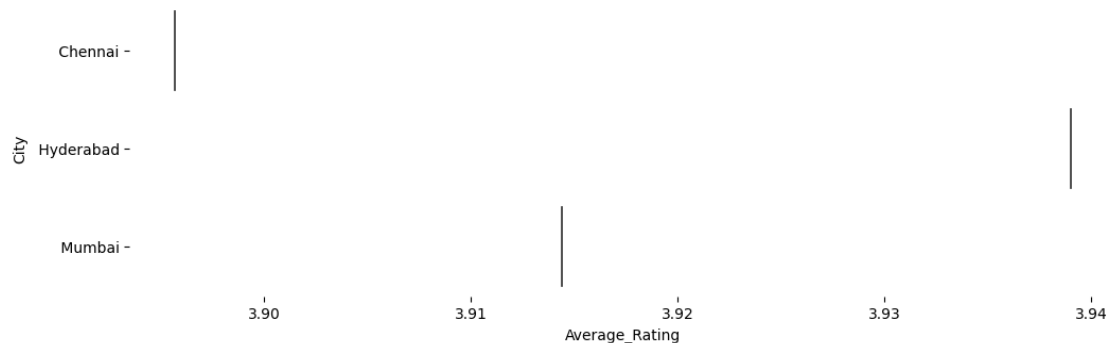
```
<google.colab._quickchart_helpers.SectionTitle at 0x7d3578a87080>
```

```
from matplotlib import pyplot as plt
import seaborn as sns
figsize = (12, 1.2 * len(_df_3['City'].unique()))
plt.figure(figsize=figsize)
sns.violinplot(_df_3, x='Average_Rating', y='City', inner='stick',
    ↪palette='Dark2')
sns.despine(top=True, right=True, bottom=True, left=True)
```

```
[ ]: from matplotlib import pyplot as plt
import seaborn as sns
figsize = (12, 1.2 * len(_df_3['City'].unique()))
plt.figure(figsize=figsize)
sns.violinplot(_df_3, x='Average_Rating', y='City', inner='stick',
    ↪palette='Dark2')
sns.despine(top=True, right=True, bottom=True, left=True)
```

```
[ ]: from matplotlib import pyplot as plt
import seaborn as sns
figsize = (12, 1.2 * len(_df_3['City'].unique()))
plt.figure(figsize=figsize)
sns.violinplot(_df_3, x='Average_Rating', y='City', inner='stick',
               palette='Dark2')
sns.despine(top=True, right=True, bottom=True, left=True)
```



```
[ ]: from scipy import stats

# Perform independent samples t-test
ttest_result = stats.ttest_ind(d['Dining_Rating'], d['Delivery_Rating'],
                               nan_policy='omit')

# Print the results
print(f"T-test statistic: {ttest_result.statistic:.2f}")
print(f"P-value: {ttest_result.pvalue:.4f}")

# Interpret the results
alpha = 0.05
if ttest_result.pvalue < alpha:
```

```

print("\nBased on the p-value, we reject the null hypothesis.")
print("There is a significant difference between the average Dining Rating_
↪and Delivery Rating.")
else:
    print("\nBased on the p-value, we fail to reject the null hypothesis.")
    print("There is no significant difference between the average Dining Rating_
↪and Delivery Rating.")

```

T-test statistic: -44.43

P-value: 0.0000

Based on the p-value, we reject the null hypothesis.

There is a significant difference between the average Dining Rating and Delivery Rating.

```

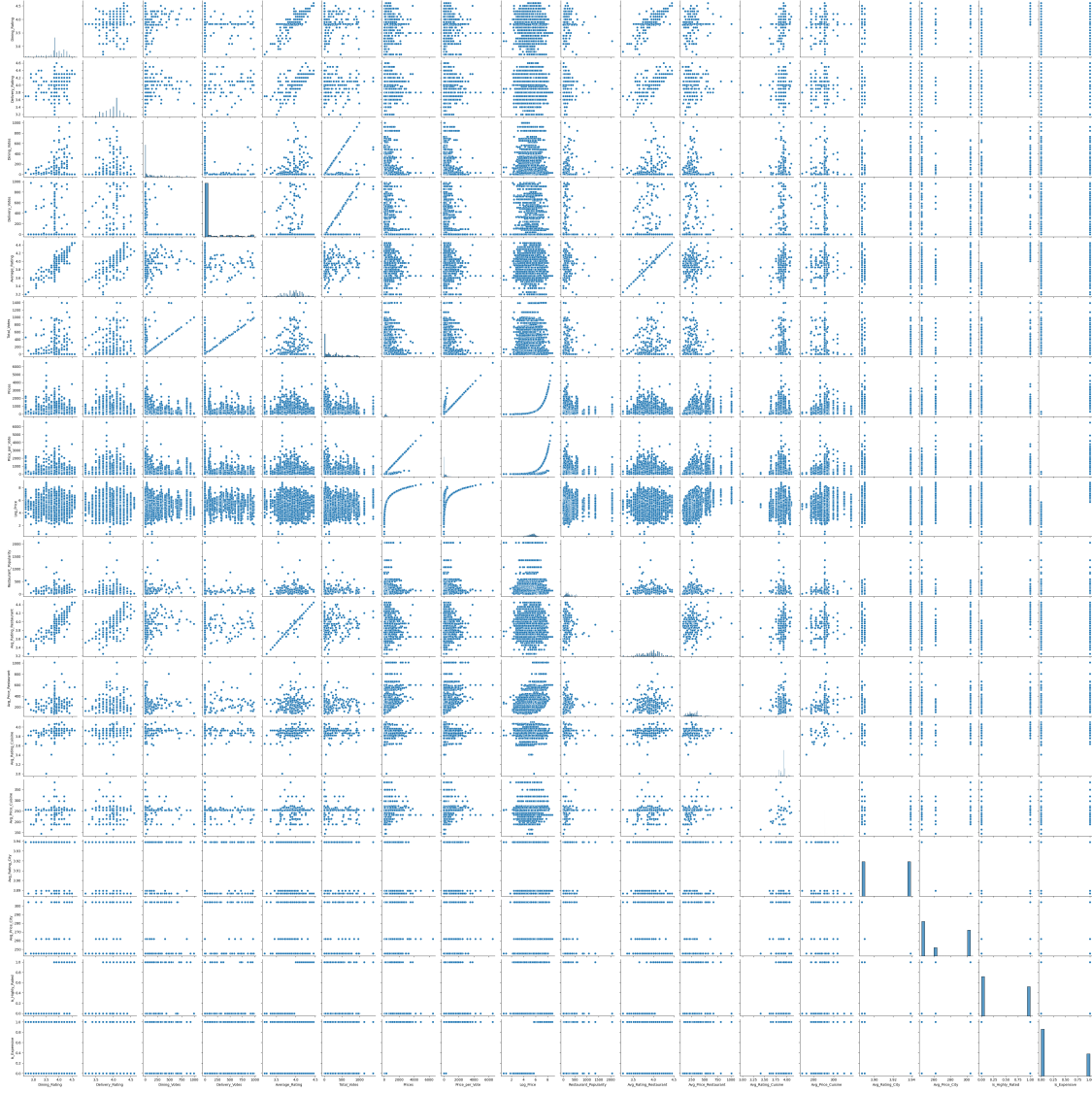
[ ]: # Select a subset of relevant numerical columns for the pair plot
relevant_numerical_cols = [
    'Dining_Rating', 'Delivery_Rating', 'Dining_Votes', 'Delivery_Votes',
    'Average_Rating', 'Total_Votes', 'Prices', 'Price_per_Vote', 'Log_Price',
    'Restaurant_Popularity', 'Avg_Rating_Restaurant', 'Avg_Price_Restaurant',
    'Avg_Rating_Cuisine', 'Avg_Price_Cuisine', 'Avg_Rating_City',
    ↪'Avg_Price_City',
    'Is_Highly_Rated', 'Is_Expensive'
]

# Ensure the selected columns exist in the DataFrame
relevant_numerical_cols = [col for col in relevant_numerical_cols if col in d.
↪columns]

# Create a pair plot with the selected numerical columns
sns.pairplot(d[relevant_numerical_cols])

# Display the plot
plt.show()

```



4.3 Business Recommendations

Based on the analysis of the Zomato dataset, here are some recommendations for identifying the next winning cuisine, pinpointing the best city/location, and defining the service model:

4.3.1 Which Cuisine to Enter:

To identify cuisine types with high ratings, strong demand, and low competition, we considered:

- **Average Rating vs. Total Votes by Cuisine and Restaurant Count:** The scatter plot helped visualize cuisines with high average ratings and total votes. By considering the size of the bubbles (representing restaurant count), we can identify cuisines with high demand but potentially lower competition (smaller bubbles).

- **Count of Restaurants by Cuisine:** The bar chart showed the distribution of restaurants across different cuisines, indicating the level of competition in each category.
- **Heatmap of Average Rating and Average Price per Cuisine:** This heatmap provided a quick overview of how different cuisines perform in terms of both rating and price.
- **Bubble Chart of Cuisine Metrics:** This chart combined average rating, average price, and total votes, offering a comprehensive view of cuisine performance and market saturation.

Recommendation: Look for cuisines that appear in the upper-right section of the scatter and bubble charts (high rating and votes) but have relatively smaller bubble sizes (lower restaurant count). Analyze the bar chart to confirm lower competition in these cuisines. For example, based on the generated plots, cuisines like “Turkish” and “Wraps” show high average ratings, and while their total votes and restaurant counts might not be the absolute highest, they could represent opportunities with less saturation compared to highly popular and competitive cuisines like “Beverages” or “Fast Food.” Further investigation into the specific price points (from the box plot and heatmap) of these potential cuisines is also recommended.

4.3.2 Which City/Area to Target:

To pinpoint cities or places with the best potential for growth, considering consumer engagement, average rating, and competition:

- **Average Rating by City:** The bar chart showed the average rating for each city.
- **Average Rating vs Total Votes by City:** The scatter plot helped visualize cities with high average ratings and total votes, indicating consumer engagement.
- **Total vs Highly Rated Restaurants by City:** The stacked bar chart provided insight into the level of competition by showing the total number of restaurants and the number of highly-rated restaurants in each city.
- **Heatmap of Average Restaurant Popularity by Place Name:** This heatmap helped identify popular places within cities.
- **Comparison of Average Price and Average Rating by City:** This chart compared average price and rating across cities.

Recommendation: Identify cities that have a good balance of high average ratings and total votes (from the scatter plot) but do not have an excessively high number of highly-rated restaurants (from the stacked bar chart). Look for places within those cities (from the heatmap of place popularity) that show high popularity but might still have room for new entrants. The comparison of average price and rating can also inform pricing strategy for the target city.

4.3.3 Which Channel to Prioritize (Delivery vs. Dining Focus):

To determine whether to focus on delivery or dining, we analyzed:

- **Dining Rating vs Delivery Rating:** The scatter plot showed the relationship between dining and delivery ratings.
- **Distribution of Dining Votes by Cuisine and Distribution of Delivery Votes by Cuisine:** The box plots illustrated the distribution of votes for dining and delivery across different cuisines.
- **Heatmap of Dining and Delivery Metrics Correlation:** This heatmap showed the correlation between dining and delivery ratings and votes.

- **Average Dining vs. Delivery Ratings by Cuisine and Average Dining vs. Delivery Votes by Cuisine:** The stacked bar charts compared average ratings and votes for dining and delivery across cuisines.
- **Average Rating vs Prices by Bestseller Status:** This scatter plot provided context on how bestseller status relates to price and rating.

Recommendation: Based on the hypothesis test, there is a significant difference between the average Dining Rating and Delivery Rating. Analyze the scatter plot of Dining Rating vs. Delivery Rating to see if one generally outperforms the other or if there are specific clusters where one channel is stronger. Examine the box plots and stacked bar charts of votes to understand which channel receives more engagement overall and for specific cuisines. The correlation heatmap can show if high dining activity is associated with high delivery activity or vice versa. Prioritize the channel that shows consistently higher ratings and votes, or focus on cuisines where a particular channel is dominant and there is market opportunity.

```
[ ]: # Select relevant numerical columns for correlation analysis
correlation_cols = [
    'Dining_Rating', 'Delivery_Rating', 'Dining_Votes', 'Delivery_Votes',
    'Average_Rating', 'Total_Votes', 'Prices', 'Log_Price',
    'Restaurant_Popularity', 'Avg_Rating_Restaurant', 'Avg_Price_Restaurant',
    'Avg_Rating_Cuisine', 'Avg_Price_Cuisine', 'Avg_Rating_City',
    ↪ 'Avg_Price_City'
]

# Ensure the selected columns exist in the DataFrame
correlation_cols = [col for col in correlation_cols if col in d.columns]

# Calculate the correlation matrix
correlation_matrix = d[correlation_cols].corr()

# Display the correlation matrix
print("Correlation Matrix of Selected Numerical Features:")
display(correlation_matrix)

# Optionally, visualize the correlation matrix using a heatmap
plt.figure(figsize=(12, 10))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Correlation Matrix')
plt.show()
```

Correlation Matrix of Selected Numerical Features:

	Dining_Rating	Delivery_Rating	Dining_Votes	\
Dining_Rating	1.000000	0.239188	0.249959	
Delivery_Rating	0.239188	1.000000	0.029147	
Dining_Votes	0.249959	0.029147	1.000000	
Delivery_Votes	-0.161866	0.028463	-0.216222	
Average_Rating	0.872559	0.683035	0.202689	

Total_Votes	0.048199	0.045818	0.559116
Prices	0.019907	0.018131	-0.000580
Log_Price	0.042650	0.007615	0.045346
Restaurant_Popularity	-0.126989	-0.066060	0.187355
Avg_Rating_Restaurant	0.809406	0.692373	0.214555
Avg_Price_Restaurant	0.047505	0.037609	-0.002479
Avg_Rating_Cuisine	0.385250	0.253200	0.047131
Avg_Price_Cuisine	0.036978	-0.044671	-0.008007
Avg_Rating_City	0.014476	0.103606	-0.057002
Avg_Price_City	0.014057	-0.110145	-0.009196

	Delivery_Votes	Average_Rating	Total_Votes	Prices \
Dining_Rating	-0.161866	0.872559	0.048199	0.019907
Delivery_Rating	0.028463	0.683035	0.045818	0.018131
Dining_Votes	-0.216222	0.202689	0.559116	-0.000580
Delivery_Votes	1.000000	-0.107439	0.688583	-0.007272
Average_Rating	-0.107439	1.000000	0.059308	0.024096
Total_Votes	0.688583	0.059308	1.000000	-0.006606
Prices	-0.007272	0.024096	-0.006606	1.000000
Log_Price	-0.012258	0.035914	0.023270	0.801093
Restaurant_Popularity	-0.133633	-0.128760	0.025676	0.040748
Avg_Rating_Restaurant	-0.102256	0.957193	0.072522	0.033084
Avg_Price_Restaurant	-0.015745	0.054656	-0.015212	0.497055
Avg_Rating_Cuisine	-0.102999	0.417181	-0.052458	-0.029122
Avg_Price_Cuisine	-0.109654	0.005341	-0.099063	0.157179
Avg_Rating_City	-0.058560	0.063015	-0.092065	-0.117107
Avg_Price_City	0.097931	-0.044842	0.076331	0.137614

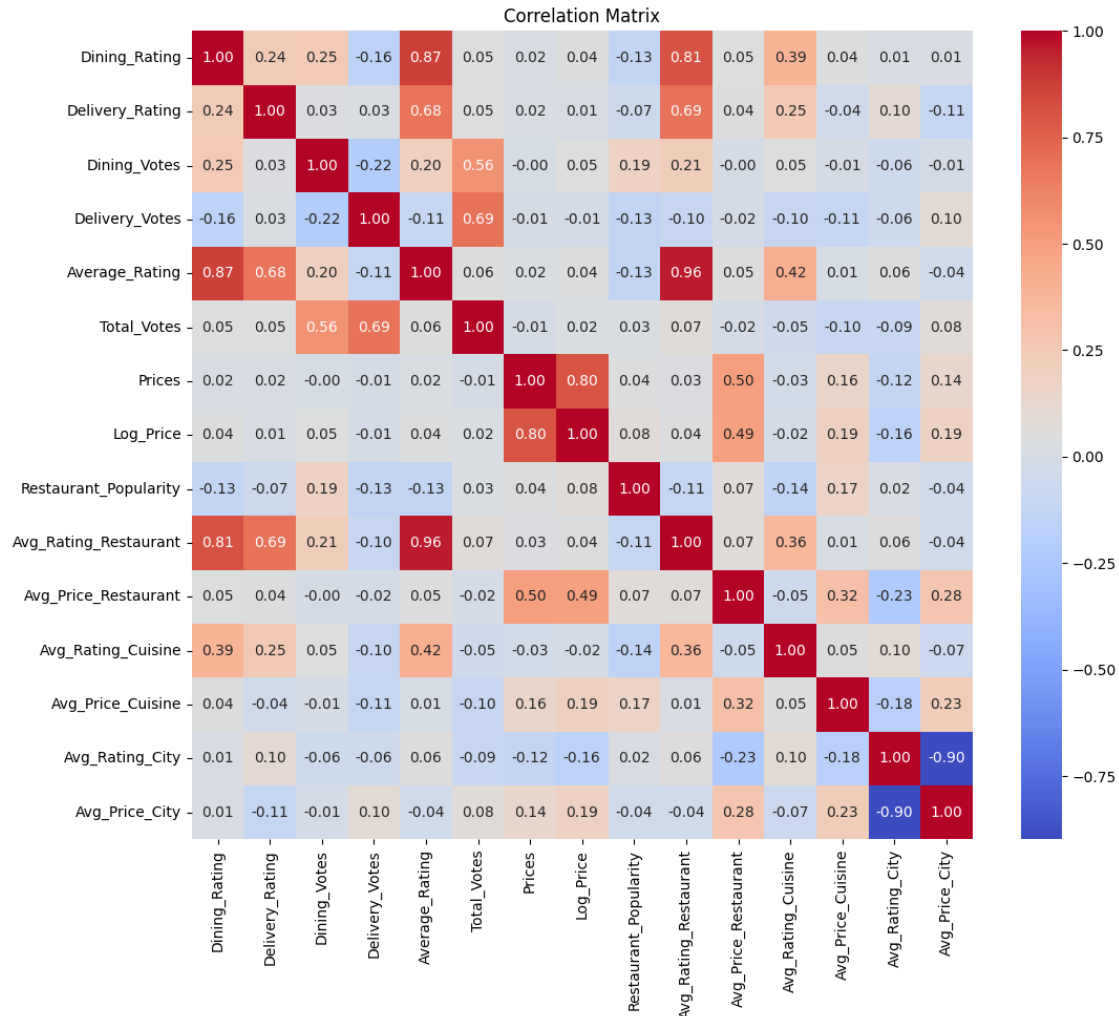
	Log_Price	Restaurant_Popularity \
Dining_Rating	0.042650	-0.126989
Delivery_Rating	0.007615	-0.066060
Dining_Votes	0.045346	0.187355
Delivery_Votes	-0.012258	-0.133633
Average_Rating	0.035914	-0.128760
Total_Votes	0.023270	0.025676
Prices	0.801093	0.040748
Log_Price	1.000000	0.078408
Restaurant_Popularity	0.078408	1.000000
Avg_Rating_Restaurant	0.042468	-0.107545
Avg_Price_Restaurant	0.492086	0.073466
Avg_Rating_Cuisine	-0.020586	-0.144051
Avg_Price_Cuisine	0.193481	0.169695
Avg_Rating_City	-0.158117	0.018261
Avg_Price_City	0.185673	-0.044568

	Avg_Rating_Restaurant	Avg_Price_Restaurant \
Dining_Rating	0.809406	0.047505
Delivery_Rating	0.692373	0.037609

Dining_Votes	0.214555	-0.002479
Delivery_Votes	-0.102256	-0.015745
Average_Rating	0.957193	0.054656
Total_Votes	0.072522	-0.015212
Prices	0.033084	0.497055
Log_Price	0.042468	0.492086
Restaurant_Popularity	-0.107545	0.073466
Avg_Rating_Restaurant	1.000000	0.068206
Avg_Price_Restaurant	0.068206	1.000000
Avg_Rating_Cuisine	0.364267	-0.053218
Avg_Price_Cuisine	0.006981	0.317663
Avg_Rating_City	0.059861	-0.234105
Avg_Price_City	-0.041310	0.276920

	Avg_Rating_Cuisine	Avg_Price_Cuisine	Avg_Rating_City \
Dining_Rating	0.385250	0.036978	0.014476
Delivery_Rating	0.253200	-0.044671	0.103606
Dining_Votes	0.047131	-0.008007	-0.057002
Delivery_Votes	-0.102999	-0.109654	-0.058560
Average_Rating	0.417181	0.005341	0.063015
Total_Votes	-0.052458	-0.099063	-0.092065
Prices	-0.029122	0.157179	-0.117107
Log_Price	-0.020586	0.193481	-0.158117
Restaurant_Popularity	-0.144051	0.169695	0.018261
Avg_Rating_Restaurant	0.364267	0.006981	0.059861
Avg_Price_Restaurant	-0.053218	0.317663	-0.234105
Avg_Rating_Cuisine	1.000000	0.051405	0.099595
Avg_Price_Cuisine	0.051405	1.000000	-0.175842
Avg_Rating_City	0.099595	-0.175842	1.000000
Avg_Price_City	-0.072006	0.229799	-0.898551

	Avg_Price_City
Dining_Rating	0.014057
Delivery_Rating	-0.110145
Dining_Votes	-0.009196
Delivery_Votes	0.097931
Average_Rating	-0.044842
Total_Votes	0.076331
Prices	0.137614
Log_Price	0.185673
Restaurant_Popularity	-0.044568
Avg_Rating_Restaurant	-0.041310
Avg_Price_Restaurant	0.276920
Avg_Rating_Cuisine	-0.072006
Avg_Price_Cuisine	0.229799
Avg_Rating_City	-0.898551
Avg_Price_City	1.000000



```
[ ]: # Display summary statistics of the DataFrame
print("Summary Statistics:")
display(d.describe())

# Check for missing values
print("\nMissing Values:")
display(d.isnull().sum())
```

Summary Statistics:

	Dining_Rating	Delivery_Rating	Dining_Votes	Delivery_Votes
count	31157.000000	31157.000000	31157.000000	31157.000000
mean	3.872342	3.976552	145.978753	117.16051
std	0.344171	0.230195	224.242799	256.38222
min	2.700000	3.200000	0.000000	0.000000
25%	3.800000	3.800000	0.000000	0.000000

50%	3.822264	4.000000	20.000000	0.00000
75%	4.100000	4.100000	209.000000	0.00000
max	4.600000	4.600000	997.000000	975.00000

	Votes	Prices	Average_Rating	Total_Votes \
count	31157.000000	31157.000000	31157.000000	31157.000000
mean	32.446545	270.047636	3.924447	263.139262
std	203.609288	217.835610	0.228770	301.918229
min	0.000000	0.950000	3.200000	0.000000
25%	0.000000	149.000000	3.800000	0.000000
50%	0.000000	235.000000	3.950000	133.000000
75%	14.000000	330.000000	4.100000	434.000000
max	9750.000000	6500.000000	4.450000	1393.000000

	Price_per_Vote	Log_Price	Is_Bestseller	Restaurant_Popularity \
count	31157.000000	31157.000000	31157.0	31157.000000
mean	178.959087	5.375763	1.0	257.297654
std	234.503975	0.698689	0.0	241.230844
min	0.007106	0.667829	1.0	5.000000
25%	12.500000	5.010635	1.0	127.000000
50%	119.900000	5.463832	1.0	199.000000
75%	280.000000	5.802118	1.0	294.000000
max	6500.000000	8.779711	1.0	2059.000000

	Avg_Rating_Restaurant	Avg_Price_Restaurant	Avg_Rating_Cuisine \
count	31157.000000	31157.000000	31157.000000
mean	3.923628	269.992525	3.898433
std	0.220328	107.983986	0.074248
min	3.250000	79.166667	3.000000
25%	3.761132	200.500000	3.862784
50%	3.950000	254.632899	3.928624
75%	4.072376	336.678899	3.928624
max	4.450000	1011.718750	4.100000

	Avg_Price_Cuisine	Avg_Rating_City	Avg_Price_City	Is_Highly_Rated \
count	31156.000000	31156.000000	31156.000000	31156.000000
mean	252.009499	3.913232	269.492607	0.427719
std	27.893303	0.025871	27.629183	0.494756
min	144.842520	3.886635	245.130482	0.000000
25%	254.494549	3.886635	245.130482	0.000000
50%	254.565364	3.939031	245.130482	0.000000
75%	255.954894	3.939031	304.402778	1.000000
max	384.209302	3.939031	304.402778	1.000000

	Is_Expensive
count	31156.000000
mean	0.320901
std	0.466830

```

min      0.000000
25%      0.000000
50%      0.000000
75%      1.000000
max      1.000000

```

Missing Values:

```

Restaurant_Name      0
Dining_Rating        0
Delivery_Rating      0
Dining_Votes         0
Delivery_Votes       0
Cuisine              0
Place_Name           0
City                0
Item_Name            0
Best_Seller          0
Votes               0
Prices              0
Average_Rating       0
Total_Votes         0
Price_per_Vote       0
Log_Price            0
Is_Bestseller        0
Restaurant_Popularity 0
Avg_Rating_Restaurant 0
Avg_Price_Restaurant 0
Avg_Rating_Cuisine   0
Avg_Price_Cuisine    1
Avg_Rating_City      1
Avg_Price_City       1
Is_Highly_Rated      1
Is_Expensive         1
dtype: int64

```

```

[ ]: # Calculate Z-scores for cuisine average ratings
cuisine_avg_rating['Avg_Rating_ZScore'] = (cuisine_avg_rating['Average_Rating'] -
    ↪ cuisine_avg_rating['Average_Rating'].mean()) /
    ↪ cuisine_avg_rating['Average_Rating'].std()

# Identify outlier cuisines (e.g., Z-score > 2 or < -2)
outlier_cuisines = cuisine_avg_rating[(cuisine_avg_rating['Avg_Rating_ZScore']
    ↪ > 2) | (cuisine_avg_rating['Avg_Rating_ZScore'] < -2)]

print("Outlier Cuisines based on Average Rating Z-score:")
display(outlier_cuisines)

```

```

# Calculate Z-scores for city average ratings
city_avg_rating['Avg_Rating_ZScore'] = (city_avg_rating['Average_Rating'] -
    ↪city_avg_rating['Average_Rating'].mean()) /
    ↪city_avg_rating['Average_Rating'].std()

# Identify outlier cities (e.g., Z-score > 2 or < -2)
outlier_cities = city_avg_rating[(city_avg_rating['Avg_Rating_ZScore'] > 2) |
    ↪(city_avg_rating['Avg_Rating_ZScore'] < -2)]

print("\nOutlier Cities based on Average Rating Z-score:")
display(outlier_cities)

```

Outlier Cuisines based on Average Rating Z-score:

	Cuisine	Average_Rating	Avg_Rating_ZScore
13	Mughlai	3.258889	-3.116218

Outlier Cities based on Average Rating Z-score:

Empty DataFrame

Columns: [City, Average_Rating, Avg_Rating_ZScore]

Index: []

5 Univariate Analysis

NUMERIC VARIABLES

Common numeric variables in your dataset:

1. Average_Rating
2. Total_Votes
3. Delivery_Votes, Dining_Votes
4. Price, Price_per_Vote
5. Log_Price

6 Histogram – Distribution of Ratings

```

[ ]: # Import libraries for visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Set figure size (width=8 inches, height=5 inches)
plt.figure(figsize=(8, 5))

```

```

# Plot a histogram of the 'Average_Rating' column
# bins=20 → divides ratings into 20 intervals
# kde=True → adds a smooth density curve over the histogram
# color='teal' → sets bar color
sns.histplot(d['Average_Rating'], bins=20, kde=True, color='red')

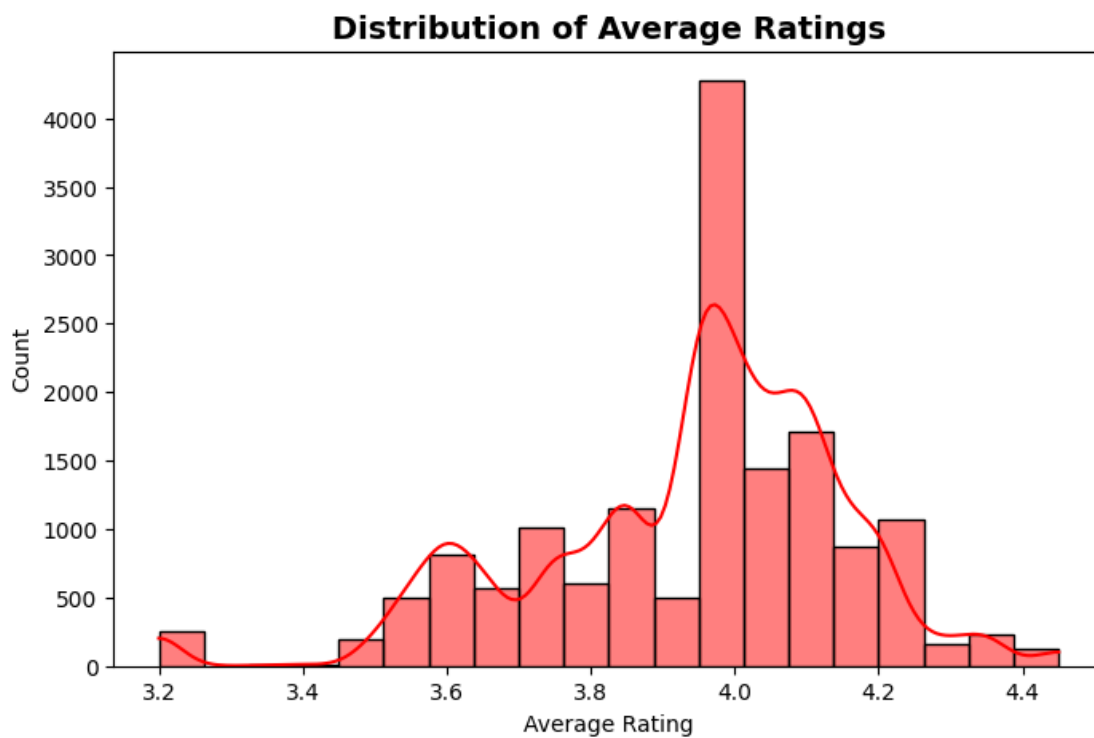
# Add a descriptive title to the chart
plt.title('Distribution of Average Ratings', fontsize=14, weight='bold')

# Label the X-axis (horizontal)
plt.xlabel('Average Rating')

# Label the Y-axis (vertical)
plt.ylabel('Count')

# Display the final plot
plt.show()

```



7 Box Plot – Price Distribution (Detect Outliers)

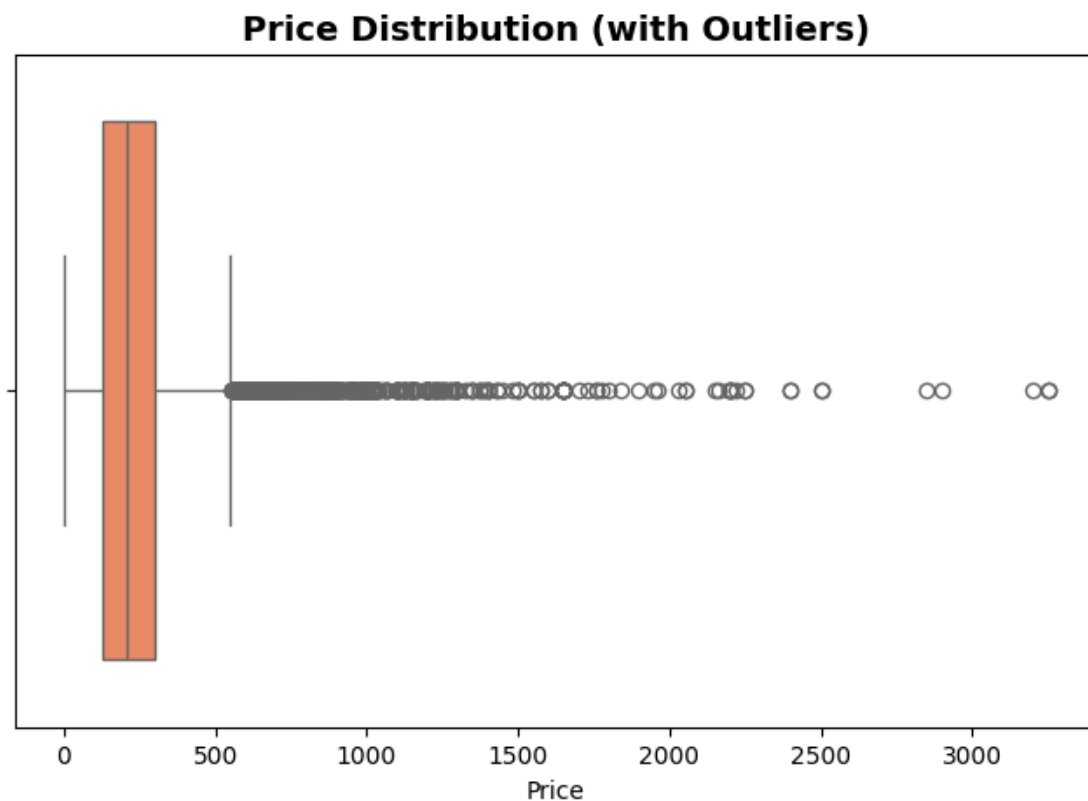
```
[ ]: # Set figure size (width=8 inches, height=5 inches)
plt.figure(figsize=(8,5))

# Create a box plot for the 'Price' column
# A box plot shows the median, quartiles, and potential outliers
# color='coral' → sets the box color
sns.boxplot(x=d['Prices'], color='coral')

# Add a descriptive title to the plot
plt.title('Price Distribution (with Outliers)', fontsize=14, weight='bold')

# Label the X-axis (horizontal axis)
plt.xlabel('Price')

# Display the final plot
plt.show()
```



8 KDE Plot – Demand (Total Votes)

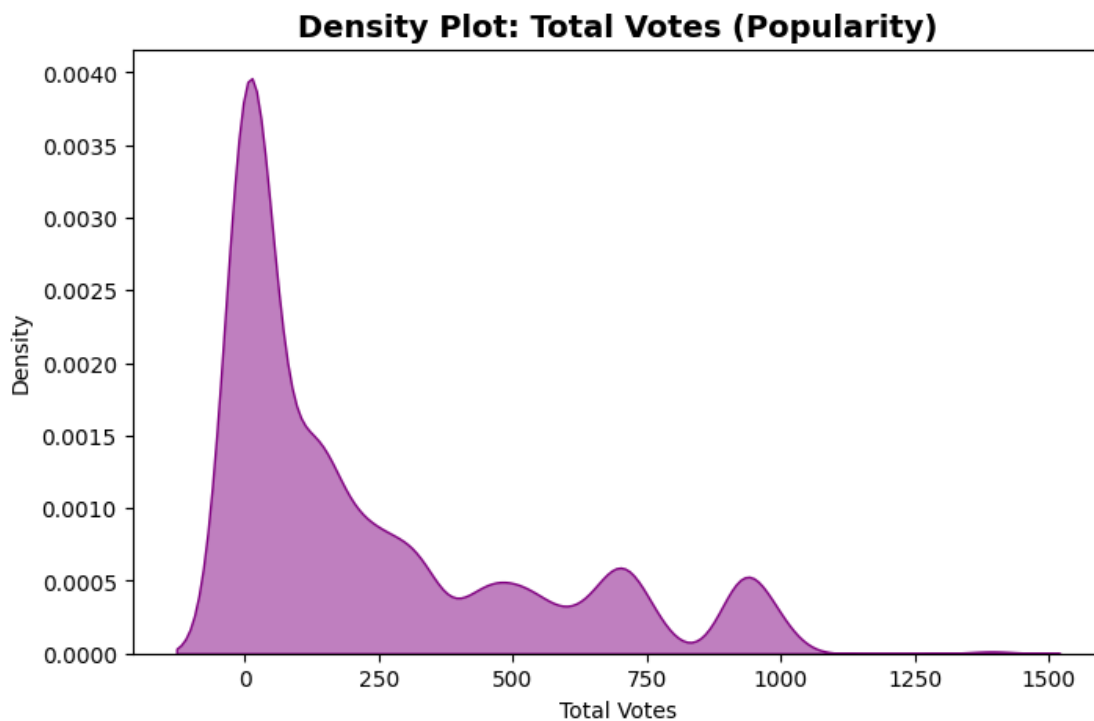
```
[ ]: # Set the figure size (width=8 inches, height=5 inches)
plt.figure(figsize=(8,5))

# Plot a KDE (Kernel Density Estimate) curve for 'Total_Votes'
# - Shows the distribution (density) of total votes instead of raw counts
# - fill=True fills the area under the curve for better visualization
# - color='purple' sets the line color
# - alpha=0.5 adds transparency so the fill isn't too dark
sns.kdeplot(d['Total_Votes'], fill=True, color='purple', alpha=0.5)

# Add a title to describe what the chart represents
plt.title('Density Plot: Total Votes (Popularity)', fontsize=14, weight='bold')

# Label the X-axis to indicate the variable being plotted
plt.xlabel('Total Votes')

# Display the final plot
plt.show()
```



9 Violin Plot – Price per Vote (Value perception)

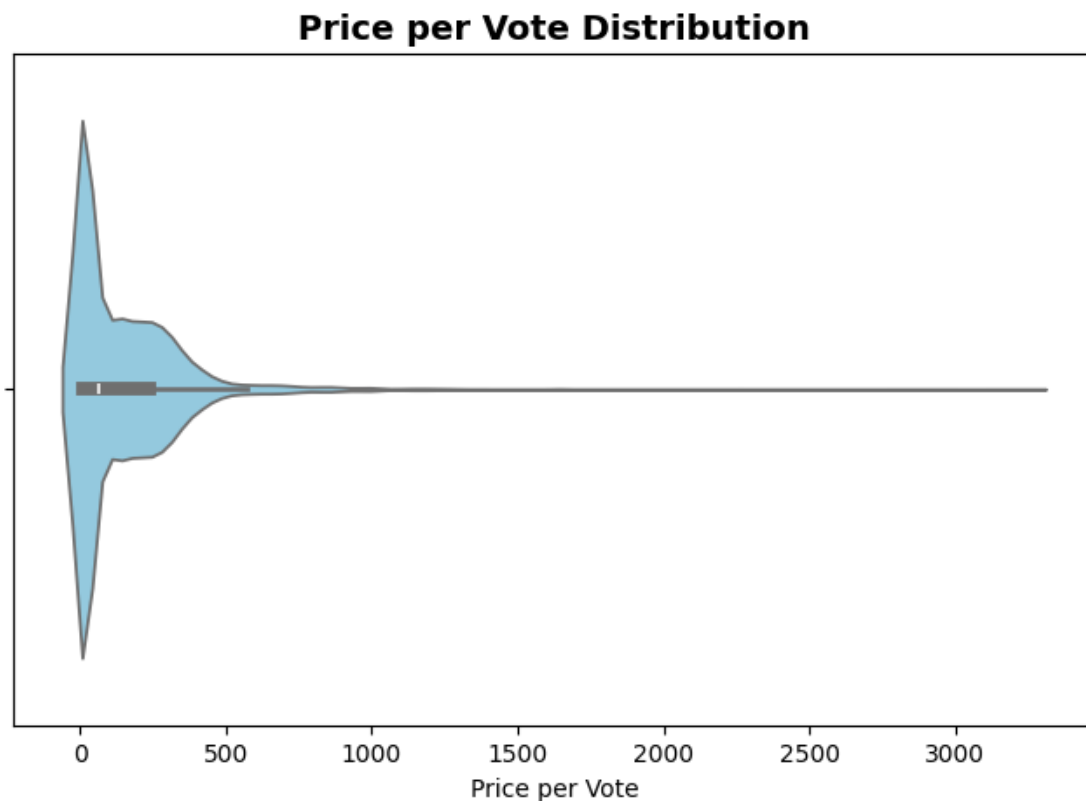
```
[ ]: # Set the overall figure size (width=8 inches, height=5 inches)
plt.figure(figsize=(8,5))

# Create a violin plot for the column 'Price_per_Vote'
# A violin plot combines a box plot and a density curve:
# - It shows the median and quartiles (like a box plot)
# - The width of the shape shows how frequently values occur (density)
# color='skyblue' sets the fill color of the violin
sns.violinplot(x=d['Price_per_Vote'], color='skyblue')

# Add a descriptive title to the plot
plt.title('Price per Vote Distribution', fontsize=14, weight='bold')

# Label the X-axis with the variable name
plt.xlabel('Price per Vote')

# Display the final plot on screen
plt.show()
```



CATEGORICAL VARIABLES

Common categorical variables:

1. Cuisine
2. City
3. Place_Name
4. Is_Highly_Rated
5. Is_Expensive
6. Channel (Dining/Delivery)

10 Count Plot – Cuisine Popularity

```
[ ]: # Set the figure size (width=10 inches, height=6 inches)
plt.figure(figsize=(10,6))

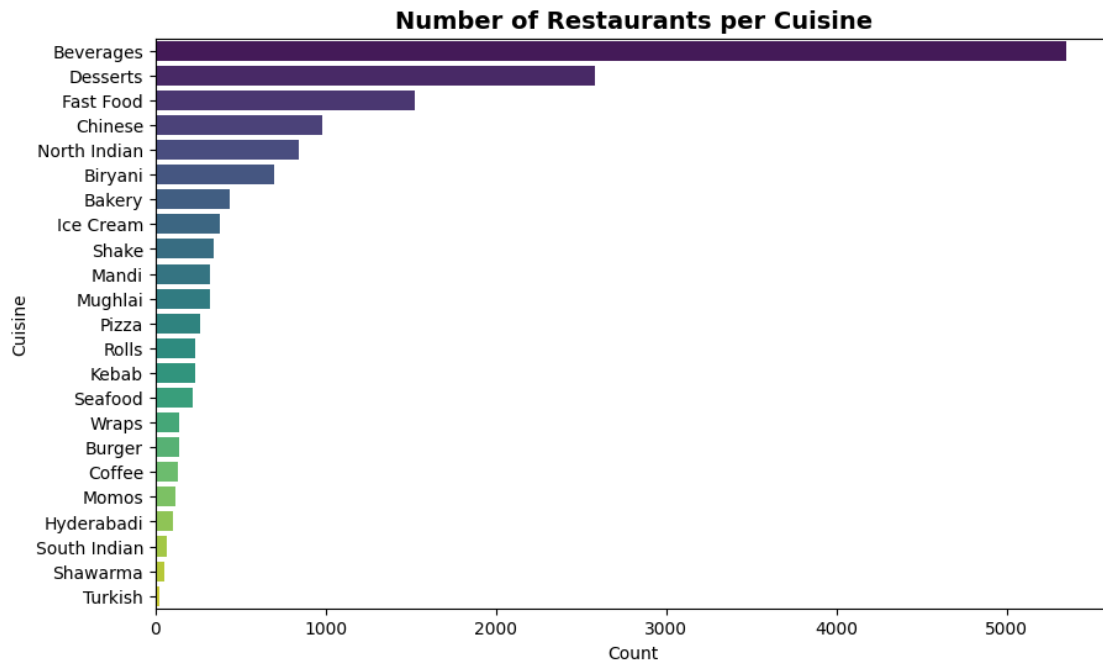
# Create a count plot to show how many restaurants exist for each cuisine type
# data=df → uses the dataframe named 'df'
# y='Cuisine' → plots each cuisine on the Y-axis
# order=df['Cuisine'].value_counts().index → sorts cuisines by frequency (most_
↳to least common)
# palette='viridis' → applies a visually appealing color palette
sns.countplot(
    data=d,
    y='Cuisine',
    order=d['Cuisine'].value_counts().index,
    palette='viridis'
)

# Add a descriptive title to the chart
plt.title('Number of Restaurants per Cuisine', fontsize=14, weight='bold')

# Label the X-axis (the numeric axis showing counts)
plt.xlabel('Count')

# Label the Y-axis (the category axis showing cuisine types)
plt.ylabel('Cuisine')

# Display the final chart
plt.show()
```

11 Count Plot – Restaurants per City

```
[ ]: # Set the figure size for better readability (width=10 inches, height=6 inches)
plt.figure(figsize=(10,6))

# Create a count plot to show the number of restaurants in each city
# data=df → uses the DataFrame named 'df'
# x='City' → plots city names on the X-axis
# order=df['City'].value_counts().index → sorts cities by the number of
# → restaurants (most to least)
# palette='coolwarm' → applies a color gradient from cool (blue) to warm (red)
sns.countplot(
    data=d,
    x='City',
    order=d['City'].value_counts().index,
    palette='coolwarm'
)

# Add a descriptive title to explain what the chart shows
plt.title('Restaurants by City', fontsize=14, weight='bold')

# Rotate X-axis labels by 45° for better readability (useful when city names
# → are long)
plt.xticks(rotation=45)
```

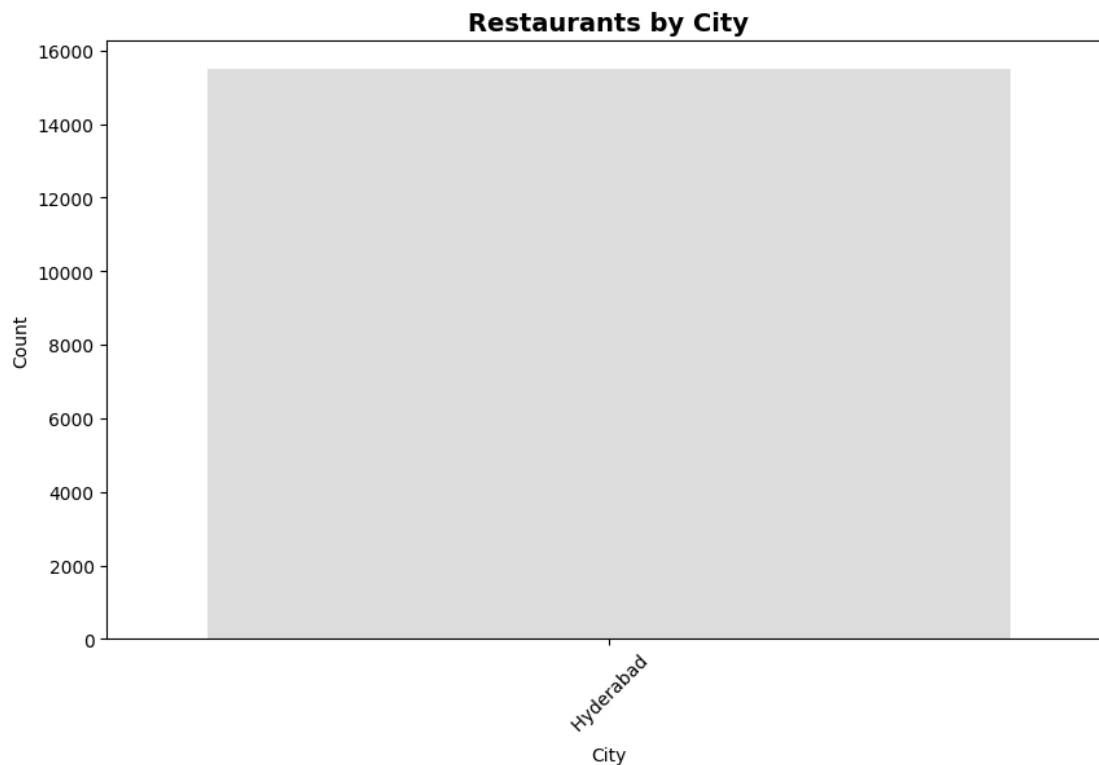
```

# Label the X-axis
plt.xlabel('City')

# Label the Y-axis
plt.ylabel('Count')

# Display the final chart
plt.show()

```



12 Pie Chart – Is_Expensive Distribution

```

[ ]: # Create a count of how many restaurants fall into each category (Expensive /
      ↳ Not Expensive)
# 'value_counts()' counts occurrences of each unique value in the
      ↳ 'Is_Expensive' column
expensive_counts = d['Is_Expensive'].value_counts()

# Set the figure size for better readability (6x6 inches for a square chart)
plt.figure(figsize=(6,6))

```

```

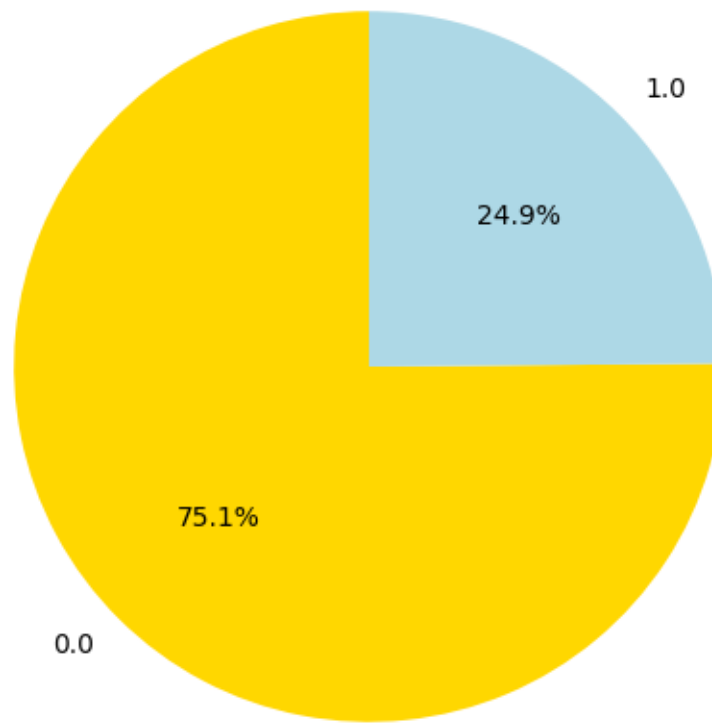
# Create a pie chart to visualize the proportion of Expensive vs Budget
↳ restaurants
# labels=expensive_counts.index → uses the category names ('Yes', 'No', etc.)
↳ as labels
# autopct='%1.1f%%' → displays percentages on each slice with one decimal place
# startangle=90 → rotates the chart so the first slice starts from the top (for
↳ a cleaner look)
# colors=['gold','lightblue'] → assigns custom colors for visual contrast
plt.pie(
    expensive_counts,
    labels=expensive_counts.index,
    autopct='%1.1f%%',
    startangle=90,
    colors=['gold', 'lightblue']
)

# Add a title to describe what the chart represents
plt.title('Expensive vs Budget Restaurants', fontsize=14, weight='bold')

# Display the final pie chart
plt.show()

```

Expensive vs Budget Restaurants



13 Bar Plot – Average Rating by Cuisine

```
[ ]: # Set the figure size for better readability (width=12 inches, height=6 inches)
plt.figure(figsize=(12,6))

# Create a bar plot showing the average rating for each cuisine
# Step 1: Group the dataset by 'Cuisine' to calculate the mean of
#         ↳ 'Average_Rating' for each cuisine
# Step 2: Reset the index (as_index=False) to turn the grouped result back into
#         ↳ a DataFrame
# Step 3: Sort the cuisines in descending order of their average rating for a
#         ↳ clear ranking
# Step 4: Pass this summarized data to seaborn's barplot
sns.barplot(
    data=d.groupby('Cuisine', as_index=False)['Average_Rating']
          .mean()
```

```

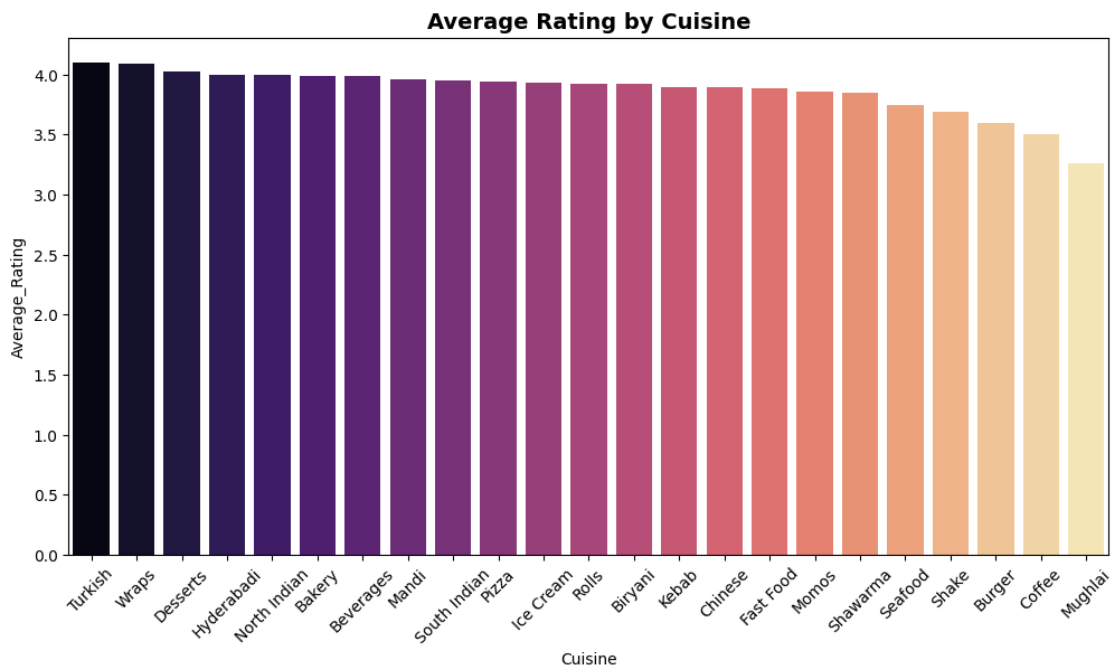
        .sort_values('Average_Rating', ascending=False),
x='Cuisine',                # Cuisines on the X-axis
y='Average_Rating',        # Average rating values on the Y-axis
palette='magma'            # Color palette for better visual appeal
)

# Add a descriptive title to the chart
plt.title('Average Rating by Cuisine', fontsize=14, weight='bold')

# Rotate the X-axis labels by 45° for better readability (especially if cuisine
↳ names are long)
plt.xticks(rotation=45)

# Display the final chart
plt.show()

```



Purpose: This chart clearly ranks cuisines by their average customer ratings. It helps us quickly see which cuisines have the best perceived quality and which ones might need improvement — valuable insight when deciding which cuisine to expand or promote.

14 Bivariate Analysis

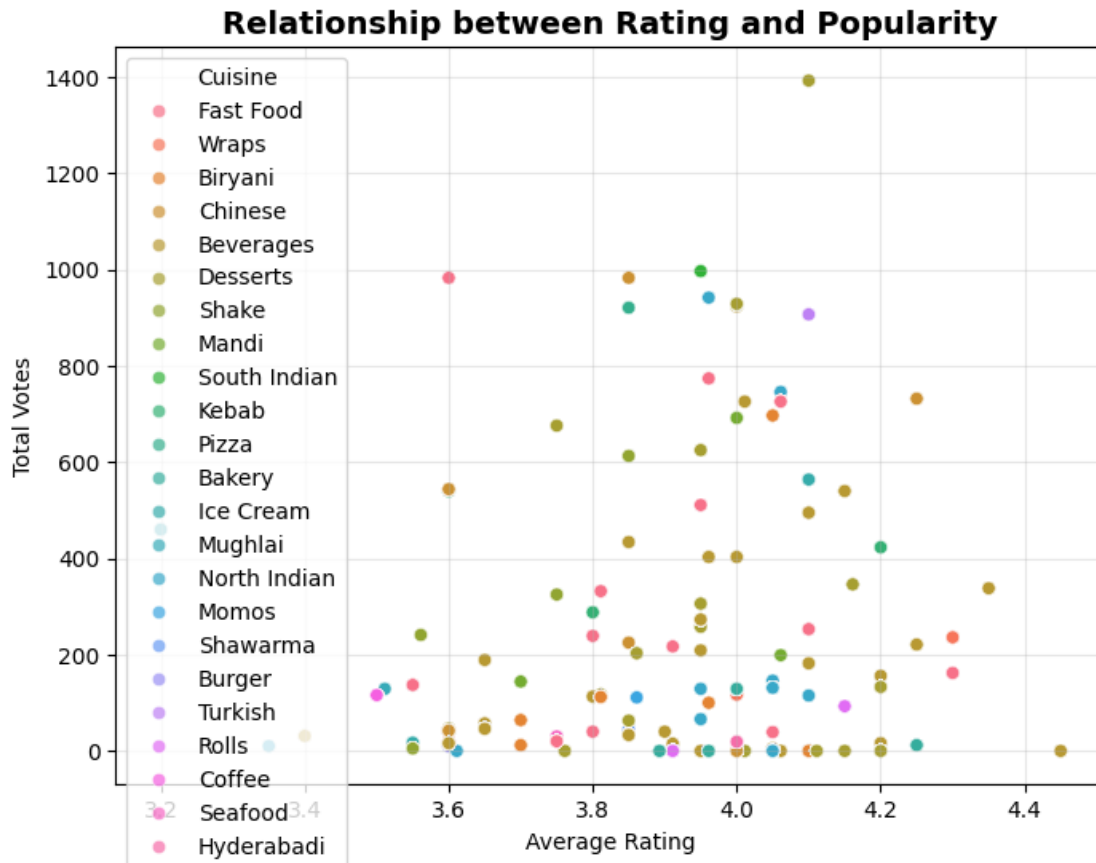
15 NUMERIC vs NUMERIC

1. Scatter Plot — Average_Rating vs Total_Votes

```
[ ]: # Set figure size
plt.figure(figsize=(8,6))

# Create a scatter plot to study how ratings relate to popularity
# x='Average_Rating' → horizontal axis (quality)
# y='Total_Votes' → vertical axis (popularity)
# hue='Cuisine' → different colors for different cuisines
# alpha=0.7 → makes points slightly transparent for better visibility
sns.scatterplot(
    data=d,
    x='Average_Rating',
    y='Total_Votes',
    hue='Cuisine',
    alpha=0.7
)

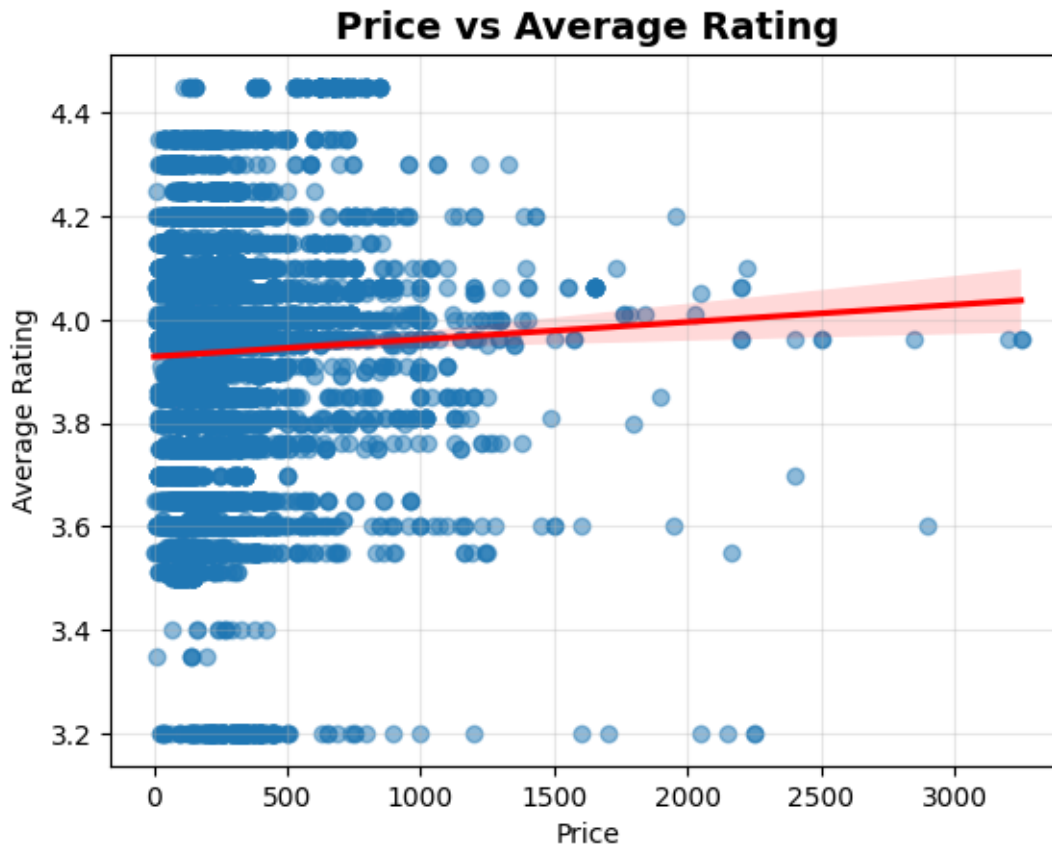
# Add title and axis labels
plt.title('Relationship between Rating and Popularity', fontsize=14,
        ↪weight='bold')
plt.xlabel('Average Rating')
plt.ylabel('Total Votes')
plt.grid(alpha=0.3) # light grid for readability
plt.show()
```



2. Regression Plot — Price vs Average_Rating

```
[ ]: # Plot a regression line (trend line)
# scatter_kws={'alpha':0.5} adds transparency to data points
sns.regplot(
    data=d,
    x='Prices',
    y='Average_Rating',
    scatter_kws={'alpha':0.5},
    line_kws={'color':'red'}
)

plt.title('Price vs Average Rating', fontsize=14, weight='bold')
plt.xlabel('Price')
plt.ylabel('Average Rating')
plt.grid(alpha=0.3)
plt.show()
```



3. Correlation Heatmap — Key Numeric Variables

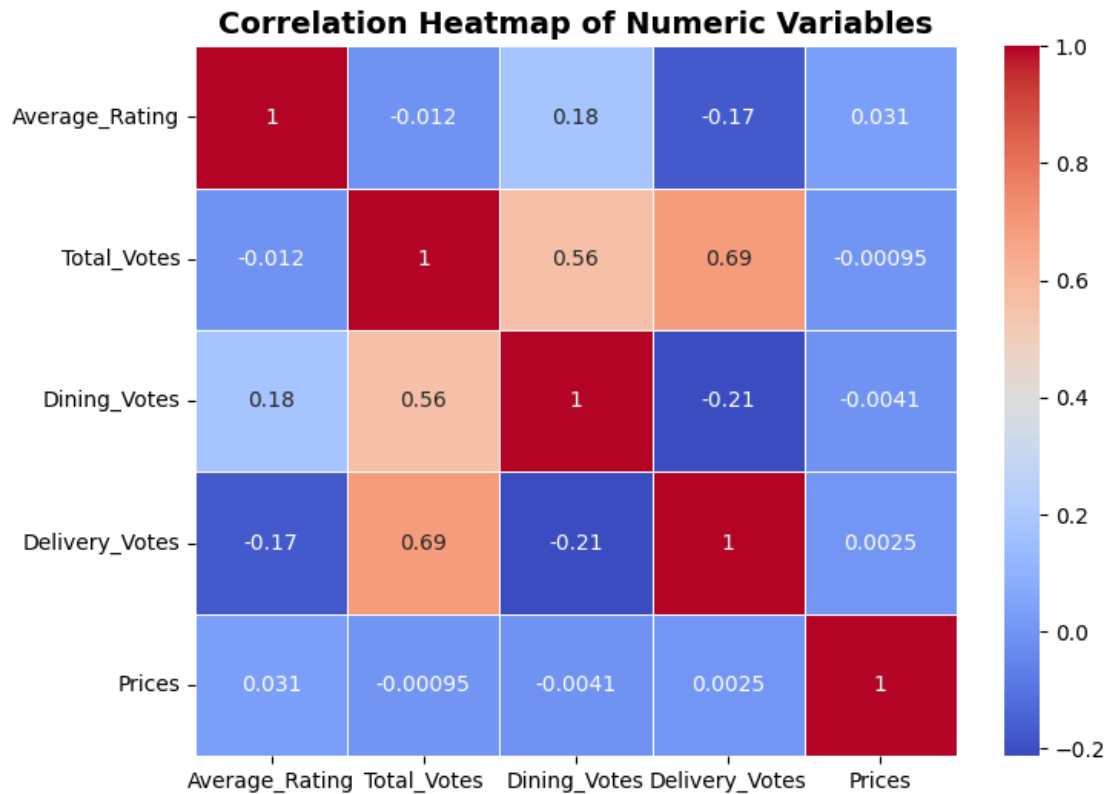
```
[ ]: # Select only numeric columns for correlation
numeric_cols = ['Average_Rating', 'Total_Votes', 'Dining_Votes',
                ↪ 'Delivery_Votes', 'Prices']

# Create a correlation matrix
corr_matrix = d[numeric_cols].corr()

# Set up figure size
plt.figure(figsize=(8,6))

# Create heatmap with annotations showing correlation values
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', linewidths=0.5)

plt.title('Correlation Heatmap of Numeric Variables', fontsize=14,
          ↪ weight='bold')
plt.show()
```

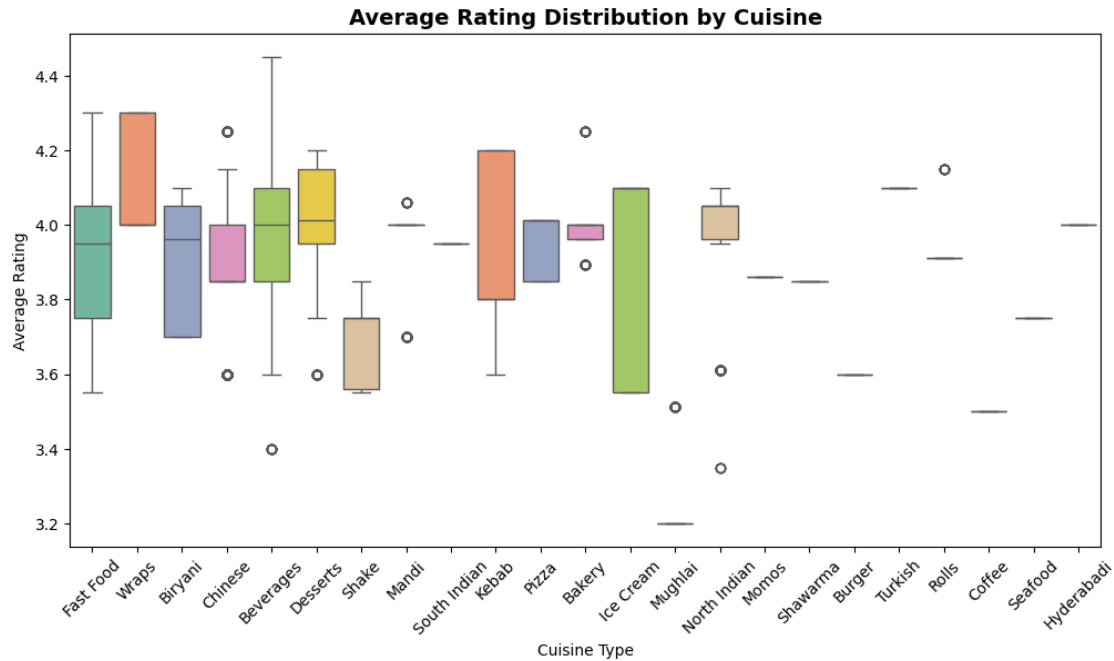
16 CATEGORICAL vs NUMERIC

4. Box Plot — Cuisine vs Average_Rating

```
[ ]: plt.figure(figsize=(12,6))

# Create a box plot to compare how ratings vary across different cuisines
sns.boxplot(
    data=d,
    x='Cuisine',
    y='Average_Rating',
    palette='Set2'
)

plt.title('Average Rating Distribution by Cuisine', fontsize=14, weight='bold')
plt.xlabel('Cuisine Type')
plt.ylabel('Average Rating')
plt.xticks(rotation=45) # Rotate X-axis labels for readability
plt.show()
```

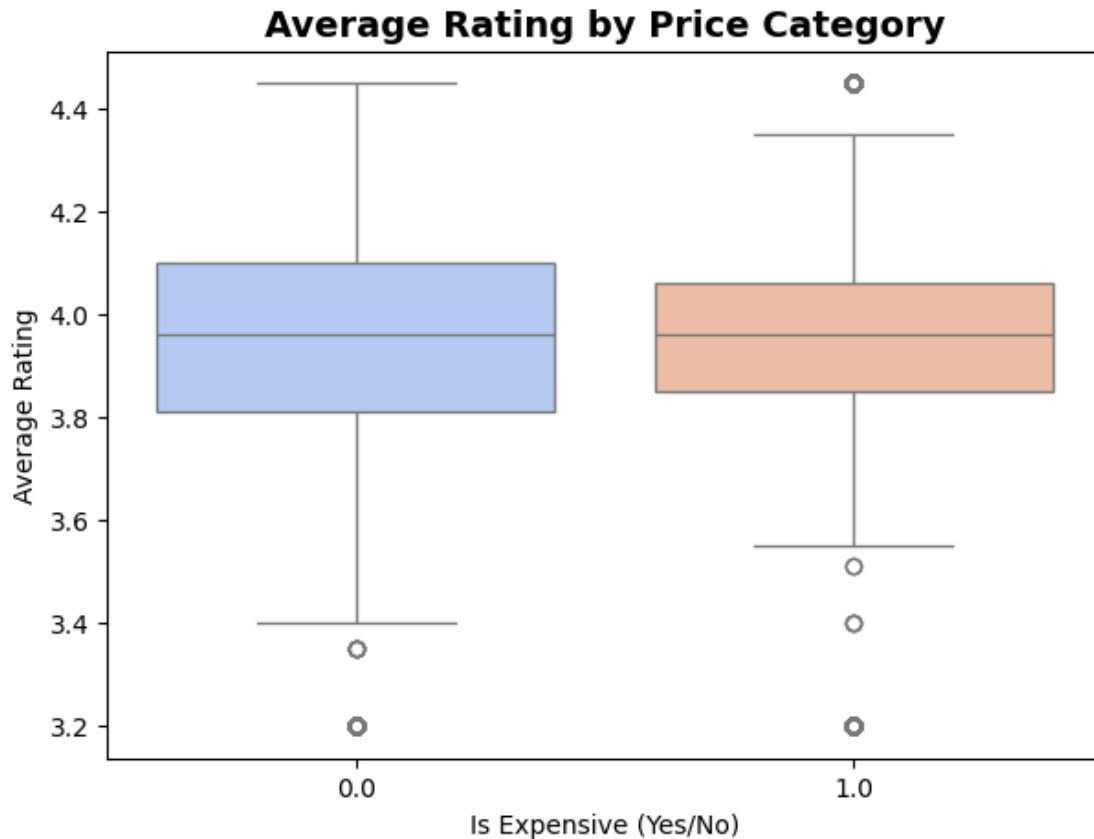


5. Box Plot — Is_Expensive vs Average_Rating

```
[ ]: plt.figure(figsize=(7,5))

# Compare ratings for expensive vs budget restaurants
sns.boxplot(
    data=d,
    x='Is_Expensive',
    y='Average_Rating',
    palette='coolwarm'
)

plt.title('Average Rating by Price Category', fontsize=14, weight='bold')
plt.xlabel('Is Expensive (Yes/No)')
plt.ylabel('Average Rating')
plt.show()
```

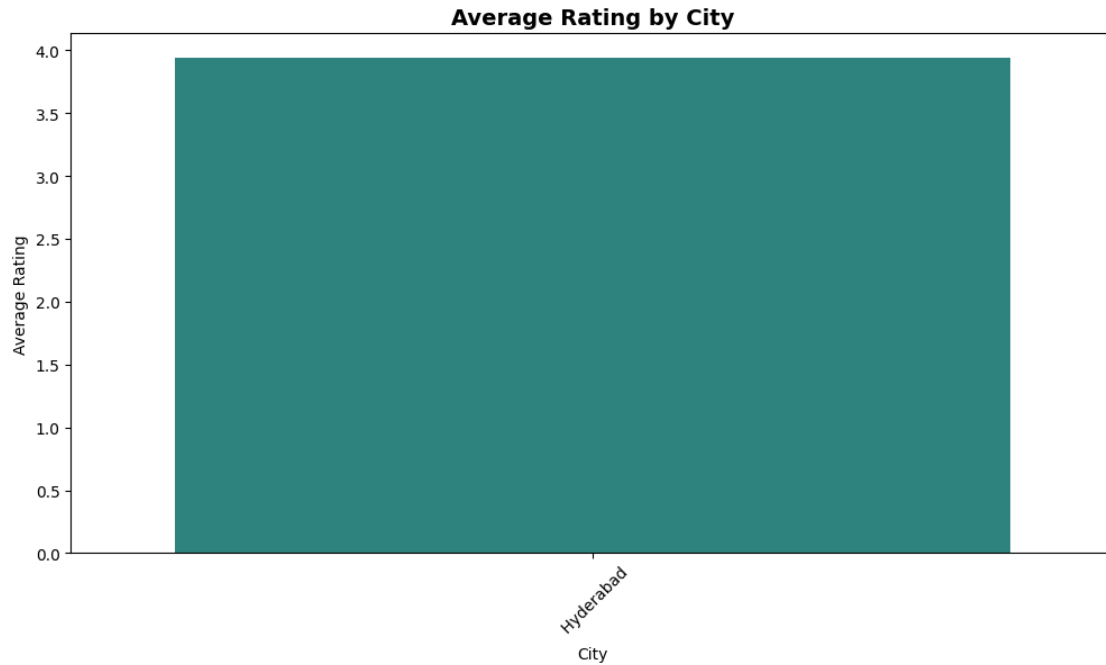


Bar Plot — City vs Average_Rating

```
[ ]: plt.figure(figsize=(12,6))

# Calculate mean rating per city and plot as a bar chart
sns.barplot(
    data=d.groupby('City', as_index=False)['Average_Rating'].mean(),
    x='City',
    y='Average_Rating',
    palette='viridis'
)

plt.title('Average Rating by City', fontsize=14, weight='bold')
plt.xticks(rotation=45)
plt.xlabel('City')
plt.ylabel('Average Rating')
plt.show()
```



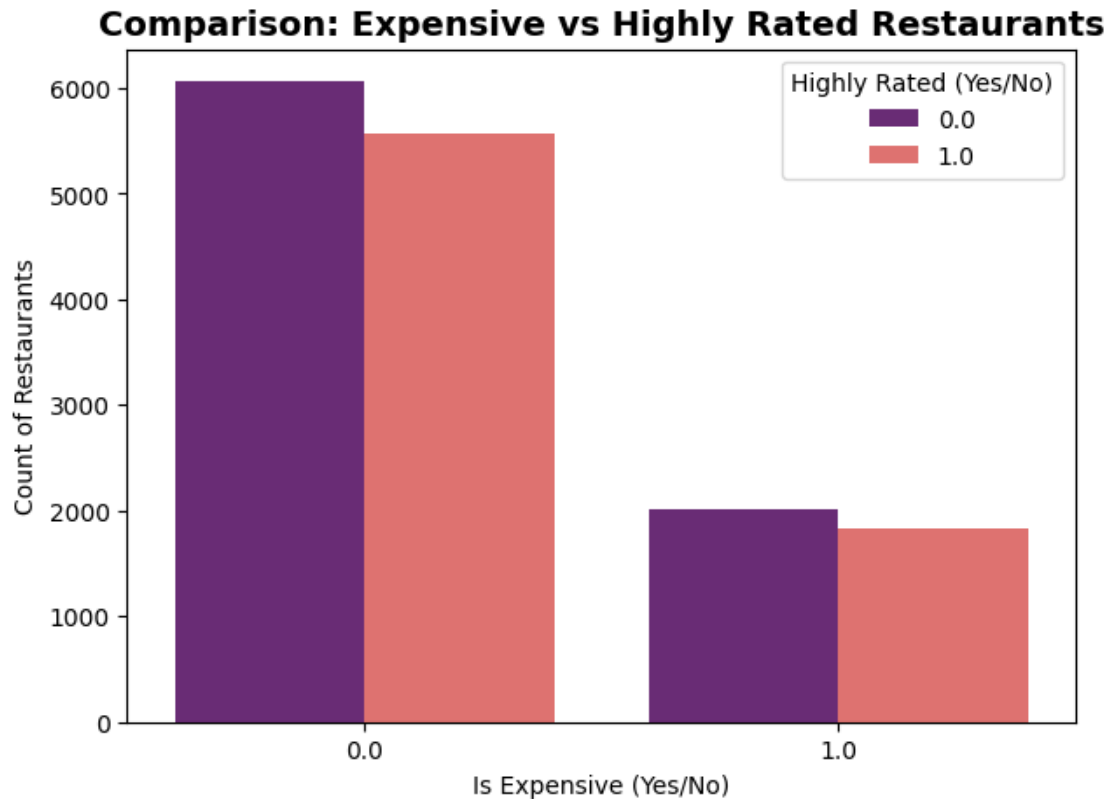
17 CATEGORICAL vs CATEGORICAL

Grouped Bar Chart — Is_Expensive vs Is_Highly_Rated

```
[ ]: plt.figure(figsize=(7,5))

# Create a grouped count plot for expensive and highly rated categories
sns.countplot(
    data=d,
    x='Is_Expensive',
    hue='Is_Highly_Rated',
    palette='magma'
)

plt.title('Comparison: Expensive vs Highly Rated Restaurants', fontsize=14,
          weight='bold')
plt.xlabel('Is Expensive (Yes/No)')
plt.ylabel('Count of Restaurants')
plt.legend(title='Highly Rated (Yes/No)')
plt.show()
```



Bivariate analysis explores the relationship between two variables to identify trends, patterns, and correlations within the restaurant dataset. It uses visual tools like scatter plots and regression plots to show how numeric variables relate — for example, whether higher prices or more total votes lead to better average ratings. Box plots and bar charts compare numeric outcomes (like ratings or votes) across categorical groups such as cuisines, cities, or price categories, helping to spot which cuisines or locations perform best. Count plots and heatmaps visualize relationships between two categorical variables, revealing patterns such as which cuisines are more common in delivery versus dining or whether expensive restaurants are usually highly rated. Overall, bivariate analysis helps uncover meaningful associations and dependencies between variables, providing insights that guide deeper analysis and better business decisions.

18 Multivariate analysis

Pair Plot (Scatter Matrix)

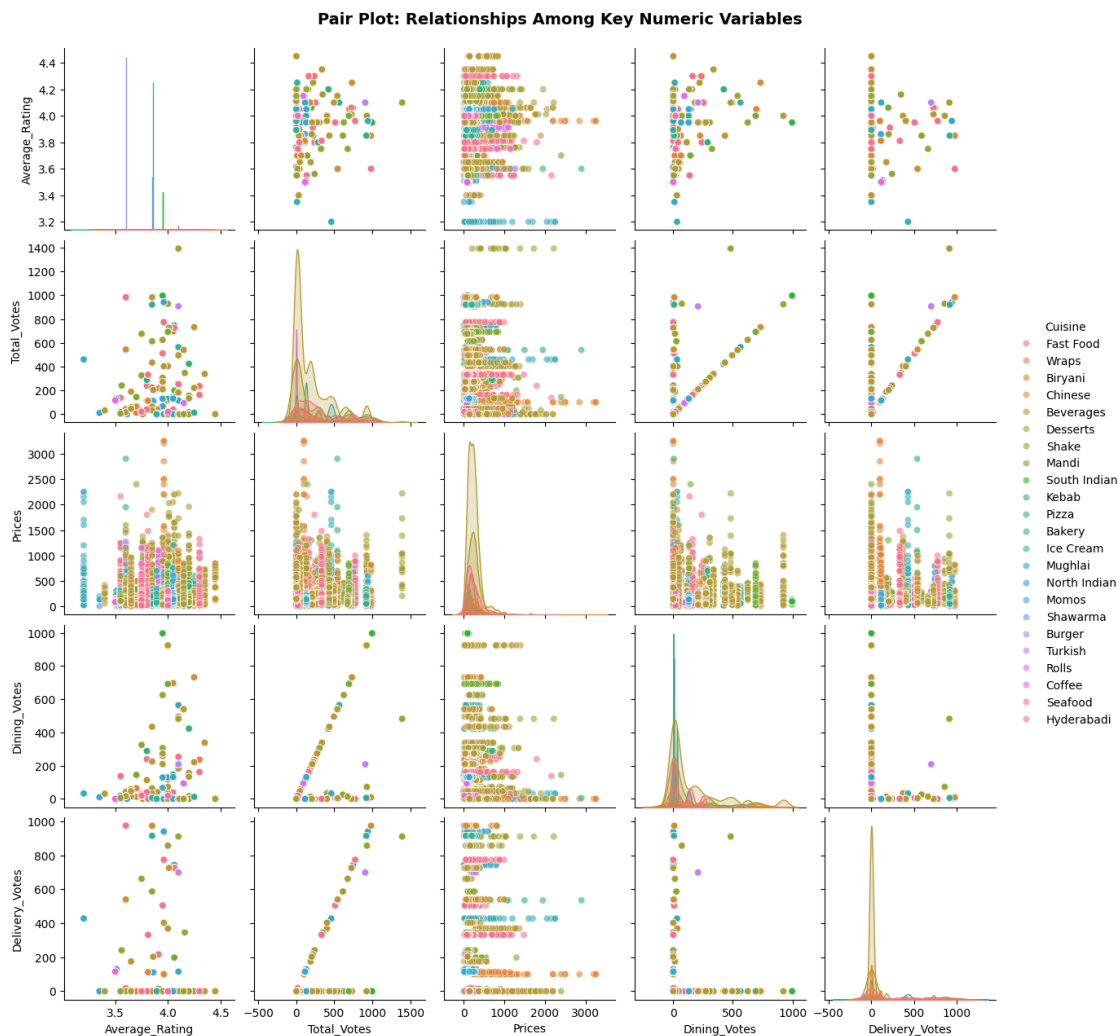
```
[ ]: # Create a pairplot to visualize relationships between several numeric columns
# hue='Cuisine' → adds color by cuisine for categorical distinction
sns.pairplot(
    d[['Average_Rating', 'Total_Votes', 'Prices', 'Dining_Votes', '
    ↪ 'Delivery_Votes', 'Cuisine']],
    hue='Cuisine',
```

```

diag_kind='kde',
plot_kws={'alpha':0.6}
)

plt.suptitle('Pair Plot: Relationships Among Key Numeric Variables', y=1.02,
            ↪fontsize=14, weight='bold')
plt.show()

```



Facet Grid (Multi-Panel Plot)

```

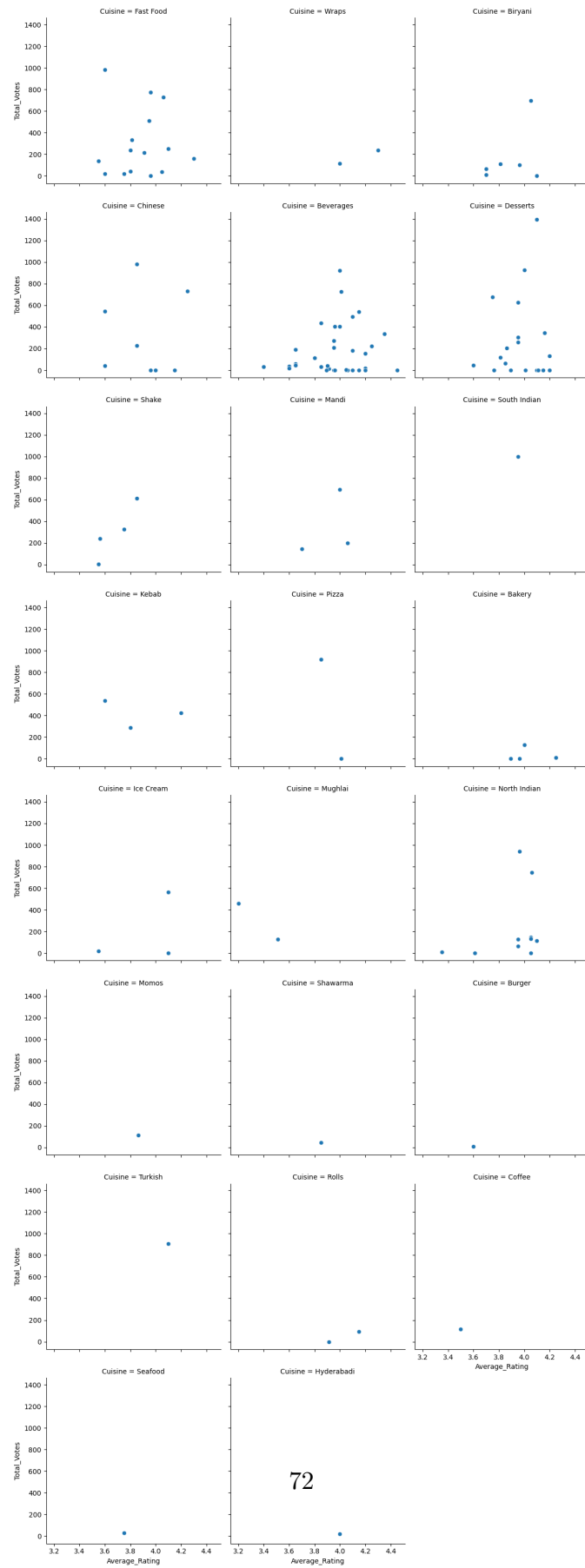
[ ]: # Use seaborn's FacetGrid to create multiple scatter plots - one per cuisine
g = sns.FacetGrid(d, col='Cuisine', col_wrap=3, height=4)
g.map_dataframe(sns.scatterplot, x='Average_Rating', y='Total_Votes', alpha=0.6)

# Add a title to the entire grid

```

```
g.fig.suptitle('Cuisine-wise Relationship between Rating and Popularity',  
↪fontsize=14, weight='bold', y=1.05)  
plt.show()
```

Cuisine-wise Relationship between Rating and Popularity



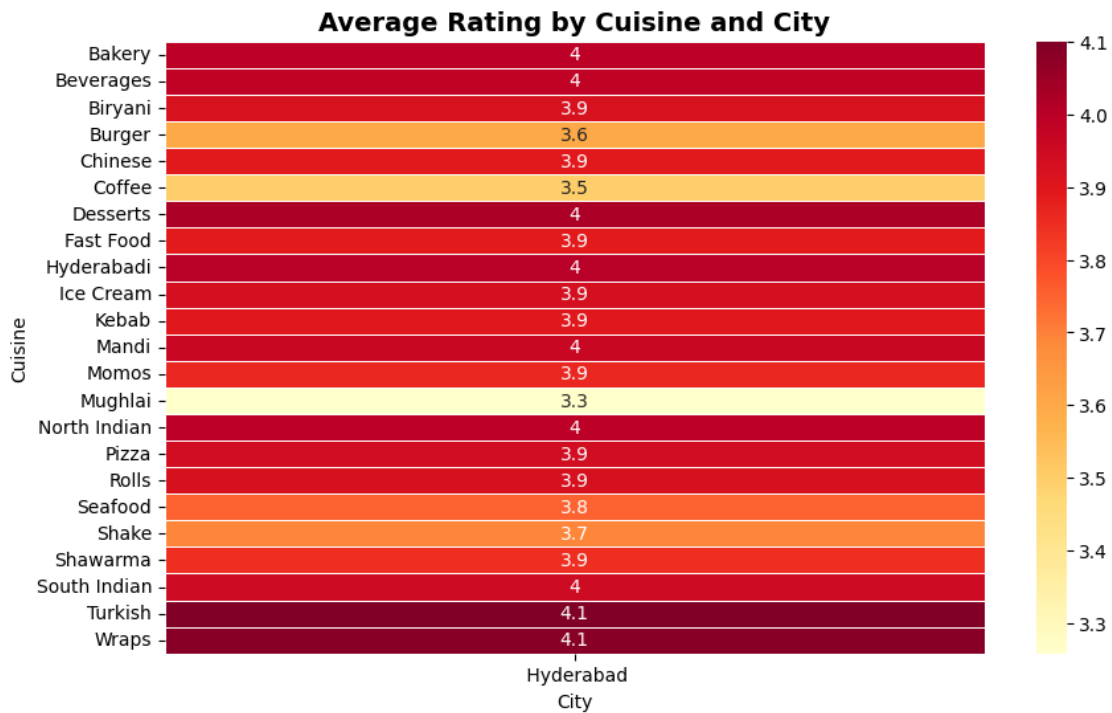
Heatmap (Cuisine vs City by Average Rating)

```
[ ]: # Create a pivot table for average ratings per cuisine per city
heat_data = d.pivot_table(
    index='Cuisine',
    columns='City',
    values='Average_Rating',
    aggfunc='mean'
)

# Set figure size
plt.figure(figsize=(10,6))

# Create a heatmap showing how ratings vary by city and cuisine
sns.heatmap(heat_data, annot=True, cmap='YlOrRd', linewidths=0.5)

# Add title and labels
plt.title('Average Rating by Cuisine and City', fontsize=14, weight='bold')
plt.xlabel('City')
plt.ylabel('Cuisine')
plt.show()
```



3D Scatter Plot

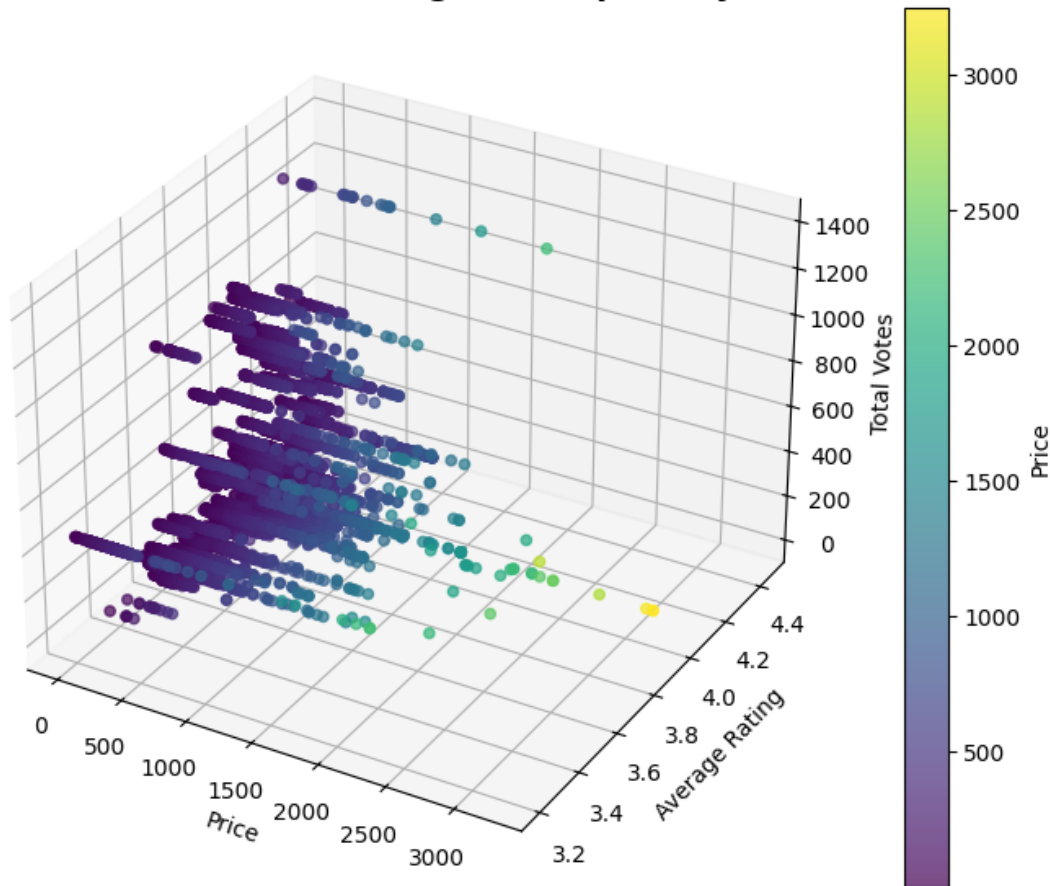
```
[ ]: from mpl_toolkits.mplot3d import Axes3D

# Create a 3D scatter plot figure
fig = plt.figure(figsize=(10,7))
ax = fig.add_subplot(111, projection='3d')

# Plot points in 3D space
# X-axis → Price, Y-axis → Average Rating, Z-axis → Total Votes
sc = ax.scatter(
    d['Prices'],
    d['Average_Rating'],
    d['Total_Votes'],
    c=d['Prices'],          # Color by price
    cmap='viridis',
    alpha=0.7
)

# Add color bar and labels
plt.colorbar(sc, ax=ax, label='Price')
ax.set_xlabel('Price')
ax.set_ylabel('Average Rating')
ax.set_zlabel('Total Votes')
plt.title('3D Plot: Price, Rating, and Popularity', fontsize=14, weight='bold')
plt.show()
```

3D Plot: Price, Rating, and Popularity



Multivariate analysis examines the relationships among three or more variables simultaneously to uncover deeper insights and patterns within the restaurant dataset. It helps identify how multiple factors—such as price, rating, cuisine, and total votes—interact and influence each other. Using visualizations like bubble scatter plots, pair plots, facet grids, heatmaps, and 3D scatter plots, this analysis reveals complex dependencies that are not visible through simple two-variable comparisons. For example, bubble plots show how popularity and ratings vary across cuisines and restaurant counts, while heatmaps highlight how certain cuisines perform better in specific cities. Clustering techniques further group restaurants with similar characteristics, making it easier to detect customer trends and business opportunities. Overall, multivariate analysis provides a comprehensive understanding of multiple dimensions together, helping make more informed and data-driven strategic decisions.